MOON-2389 Client Operating Model — Backend Architecture

Overview

A short, non-technical summary of MOON-2389. The other tabs hold the engineering detail.

The idea in one paragraph

Every client runs projects their own way: their own steps, their own rules, their own people in specific roles. Today Veda gives everyone the same generic guidance. With this feature a client describes how they work once, in Veda, and from then on the AI suggests the right next step to the right person at the right moment, following that client's way of working instead of a generic one.

The three building blocks

Building block	What it is	Example	Who manages it
Project Team Roles	The part someone plays on a project. Not the same as their access level.	PM, Business Analyst, Designer	Company admins keep the list; project admins assign roles to team members
Policies	A reusable rule: what should happen, optionally when, and the activities it involves.	"New Project Kickoff": when a project starts, schedule the kickoff meeting and post an update to Slack	Company admins
Methodologies	The client's way of running a project, as ordered stages. Each stage holds activities and can include policies.	"Enterprise Delivery Framework": Initiation → Discovery and Scope → Delivery → …	Company admins build them in a visual editor (Figma)

An **activity** is one piece of expected work, such as "send the kickoff email" or "create the RAID record". It says which roles it is for and, optionally, which Veda agent helps with it.

How it works on a real project

1. A company admin sets up roles, policies and a methodology, then **publishes** it. Edits stay in a draft until it is published. Whether publishing changes projects that already use the methodology is still being decided.

2. A project admin picks the methodology for the project in Project Settings (Figma) and gives team members their roles.
3. As the project moves along (a call happens, a deadline nears, project health turns red), the AI team's orchestrator reads the methodology and policies and decides what should happen next.
4. It sends a **suggestion** to the people in the right role, for example "Post a Slack update about scope sign-off" to the PM.
5. A person reviews it and applies or dismisses it. Applying runs the action, such as posting the message.
6. When the AI thinks the project is ready for the next stage, it suggests the move; a project admin confirms it, and the Overview stage card (Figma) shows the new stage. Whether stages are in the first version is still being decided.

People stay in control. The AI only suggests; it never changes a project on its own in the first version.

Who does what

Team	Responsible for
Uinno (backend and frontend)	Storing roles, policies and methodologies; the admin screens and the editor; Project Settings; delivering suggestions to the right people; the agreed contract with the AI team
AI team	Deciding when something should happen, interpreting policies, writing the suggestions, and later reading methodologies out of uploaded documents

What ships when

	Included
First version	Role library and role assignment; policy library; methodology editor; choosing a methodology for a project; suggestions sent to the right roles
Possibly in the first version (still being refined)	Publish and versions; document upload (stored only); the project's current stage and the stage card; a full change history (audit); the connection the AI team uses to read this data and send suggestions
Later	Tracking which activities are done; recording why a required step was skipped; an adherence score; letting the AI act without approval; AI-proposed changes to methodologies; reading methodologies from documents; Suggestion Sets

Where to read more

- Architecture: how it is built, for engineers.
- AI team contract: what we agree with the AI team.
- Open questions: decisions still open.
- Future phase: what comes after the first version.

Architecture

Oct 26, 2026 · @Artem Derevyanko

Roles, policies and versioned methodologies live in integration-sync-service and reach the AI orchestrator through a scoped internal API; the AI team detects triggers and writes role-targeted suggestions back into our suggestion store.

Decisions at a glance

Eleven decisions shape everything below. Each is a recommendation; the ones that depend on product input also appear in the Open questions tab.

#	Decision	Why
1	Use **Stage** and **Activity** everywhere. Tables are prefixed `client_methodology_*`, `client_policies*`, `client_activities`, `project_team_role*`.	Meeting on 20 Oct renamed Gate ⟶ Stage and Action ⟶ Activity, and Figma follows it. The prefixes avoid `collection_methodology`, `llm_policies*`, `subscriber_role_hierarchy` and `project_stage`, which already exist.
2	An Activity belongs to exactly one container: a Policy or a Stage. A Stage references 0..N Policies and also holds its own direct Activities.	The methodology editor in Figma shows both in Stage 1 "Initiation": the "New Project Kickoff" policy block and a direct "Identify stakeholders" activity. Single ownership removes the copy and origin-tracking logic MOON-2394 describes.
3	Methodologies are edited as a draft and **published** into an immutable version snapshot. Projects pin a version id. Policies keep an append-only revision history.	A project must keep the version it was given (MOON-2393). A policy edit must not rewrite history (MOON-2389). How projects pick up a new version is still open question #9.
4	The editor saves the whole tree in one call: PUT `/client-methodologies/{id}/structure`. A plpgsql RPC applies it atomically with an optimistic revision check.	The canvas has undo/redo and a single Save button. ISS has no Python transactions (supabase-py only), so atomic multi-row writes must be RPCs.
5	One always-on, append-only `operating_model_audit_trail` table is both the audit log and the outbox for the AI event feed.	`tenant_audit_trail` skips service-role writes, is opt-in per tenant and is pruned after 30–180 days.
6	Project Team Roles are an org library. A member's roles hang off their `project_assignment` row. Users get a	Matches MOON-2390. A real FK with ON DELETE CASCADE means removing a member removes their roles, with no cleanup trigger. A project

#	Decision	Why
	default role for prefill. Deleting a role cascades to its assignments and activity links; the UI asks for confirmation first.	owner with no assignment row holds no roles until someone adds them to the team; no row is created implicitly.
7	Project runtime state lives in a new table: the current stage. A state per activity comes later (Future phase). Possibly in scope: open questions #19, #20 and #29.	Figma shows the stage bar, and the orchestrator needs to know which stage's activities apply. `project.project_stage` is a fixed Pre-Sale / Delivering / Inactive enum and cannot hold this. Deviations, adherence and manual marking are a future phase (see the Future phase tab).
8	Orchestrator suggestions reuse the existing suggestion store. We add assignees, source attribution and one typed suggestion for stage moves, `project.methodology.change_stage`.	Suggestions are stored and applied in ISS today. The Figma activity types already map onto existing executable suggestion types.
9	The AI team gets a separate `/internal/orchestration/*` router behind a scoped service key.	Today's single `X-API-KEY` reads every tenant with all feature flags on.
10	Methodology documents are stored by ISS in Supabase Storage. The file type is checked by content, not by the name or the client MIME. pathfinder `/textextract` is used later for parsing.	ISS already has `FileStorage` and upload guards. pathfinder's storage keys are not unique and it trusts the client MIME type.
11	New operating-model writes that span several rows or tables, need a transaction, or must produce an event for the AI team go through ISS APIs. A single field the browser already writes stays a direct Supabase write.	Validation, audit and events need one write path. The project team (people, access levels, roles) meets all three tests and moves to ISS: `PUT /projects/{id}/team`. `users.default_project_team_role_id` meets none: it is one column, guarded by RLS and a composite FK, so the frontend writes it to Supabase. Direct writes to `project` and to agent assignments stay as they are.

Scope and terminology

Our team builds storage, APIs, UI, suggestion delivery and the contracts. The AI team builds the orchestrator, which detects triggers, interprets policies, writes suggestion content and parses documents.

Glossary

Term	Meaning	Stored in
Methodology	An org-owned operating model, e.g. "Enterprise Delivery Framework v2". Type is One-time or Recurring.	`client_methodologies`
Methodology version	An immutable published snapshot of a methodology. Projects pin one.	`client_methodology_versions`
Stage	An ordered step of a methodology ("Stage 1 · Initiation"). Called Gate in older tickets.	`client_methodology_stages`
Policy	A reusable rule: name, description, optional trigger and reason, plus its own activities. Either standalone or attached to stages.	`client_policies`
Activity	A unit of expected work: type, description, project roles, agent, prompt, required flag.	`client_activities`
Project Team Role	The functional role someone plays on a project (PM, BA, Designer). It is not an access level.	`project_team_roles`
Current stage	Where a project is within its pinned methodology version.	`project_methodology`

Names already taken

Word	Existing meaning	How we avoid confusion
Stage	`project.project_stage` is the Pre-Sale / Delivering / Inactive enum, shown as "Status". The Certinia stage ("7/8 Negotiation/Review") is read from Salesforce.	Always say "methodology stage" in code and API. The UI label is open question #19.
Role	"Access Roles" is `subscriber_role_hierarchy` (`users.role`). "Profiles" is `subscriber_user_profiles`. Project access level is Viewer 100 / Contributor 200 / Editor 300 / Administrator 400.	Table, API and UI label are "Project Team Role"; the API path is `/project-team-roles`.

Word	Existing meaning	How we avoid confusion
Policy	`llm_policies*` (LiteLLM guardrails) and `auth_ip_policy`.	Prefix `client_policies`. Path `/client-policies`, never `/llm/policies`.
Methodology	Enum `collection_methodology` (sprint_based / continuous) on ticketing boards.	Prefix `client_methodology`.
Agent	`query_form` rows are Veda agents (Emily, Cosmo). `relay_agents` are suggestion prompt profiles.	`client_activities.agent_id` references `query_form(form_id)`, as `project_assignment.assigned_agent` already does.
Action	Reserved by the wider orchestration project.	Not used.

In scope

- Org role library, project role assignments, user default role.
- Policy library with activities, revisions and usage. Archive depends on open question #8.
- Methodology CRUD and canvas save. Publish and versions depend on open question #9; document upload on #14.
- Project methodology selection. The current stage is possibly in scope (open questions #19, #20).
- Suggestion store changes: assignees, source attribution, `project.methodology.change_stage`.

Possibly in scope

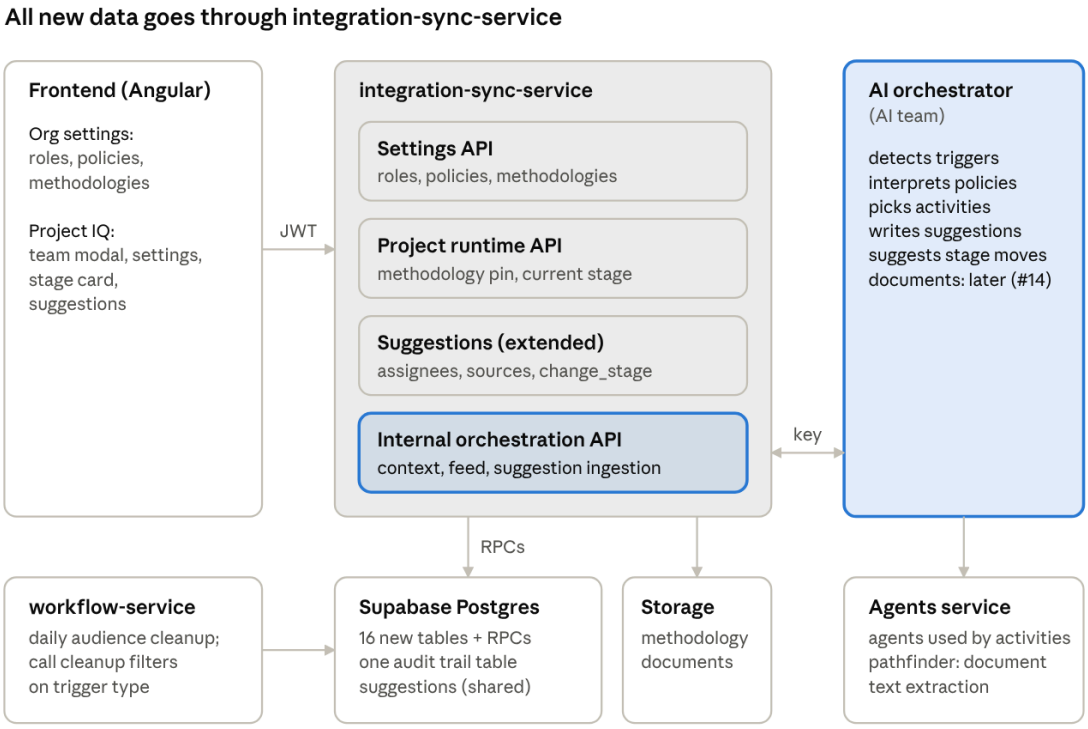
- Internal orchestration API, event feed, scoped service auth.
- Audit for every change in scope.

Out of scope (AI team or later)

- Trigger detection, policy interpretation and suggestion content generation.
- Methodology extraction from documents and the "Engage with Agent" model.
- Autonomous execution without a human approving it.
- Deviations, the stage-advance check, adherence, manual activity marking, orchestrator autonomy, AI proposals and Suggestion Sets: see the Future phase tab.

System overview

Four new areas in integration-sync-service own every write. The AI orchestrator never touches the database: it reads context and the event feed, and writes suggestions and stage moves through the internal API.

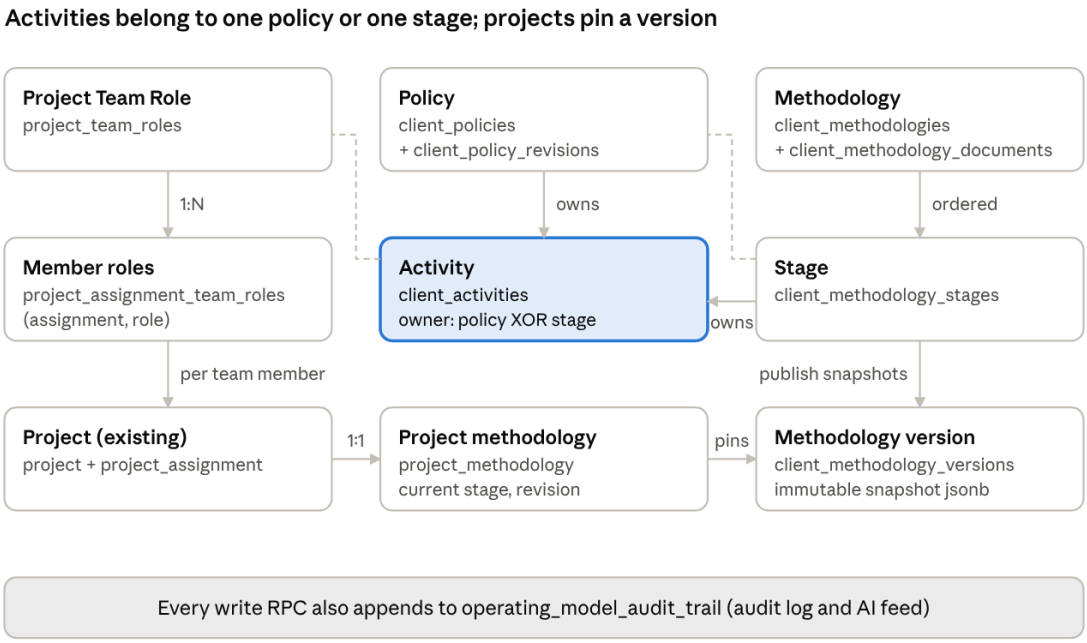


system overview · 4 ISS modules, AI team through the internal API only

Two existing paths stay. The frontend still writes `project`, agent assignments and the new `users.default_project_team_role_id` column straight to Supabase, and DB triggers turn owner changes into events. People on the project team go through ISS (`PUT /projects/{id}/team`). suggestion-service keeps generating call suggestions unchanged.

Domain model

The model adds 16 tables to the shared Supabase Postgres, plus one `suggestions` column, one `users` column and a `UNIQUE (tenant, form_id)` constraint on `query_form`. Migrations live in `integration-sync-service/migrations/sprint-NN/`.



entity diagram · library, roles and project runtime

Dashed lines are join tables. Stage to Policy goes through `client_methodology_stage_policies`, and Activity to Role goes through `client_activity_roles`. Not drawn: `audiences`, `audience_members` and `service_credentials`.

Conventions for every table

- `id uuid PK default gen_random_uuid()`.
- `tenant bigint NOT NULL REFERENCES subscriber(tenant)`.
- Audit columns `created_at`, `created_by`, `updated_at`, `updated_by`. Append-only tables only have the first two.
- **RLS**: enabled on every table, with a RESTRICTIVE `tenant_isolation` policy and **no permissive policy**. The tables are reachable only through ISS, which uses the service role, as `ticketing_ticket_events` already is.
- **RPCs**: `SECURITY INVOKER`, followed by `REVOKE ALL ON FUNCTION … FROM PUBLIC, anon, authenticated` and `GRANT EXECUTE … TO service_role`. Existing migrations already revoke from PUBLIC. A browser JWT therefore cannot call `/rpc/publish_client_methodology` with a forged actor.
- **Triggers** fire on browser writes to `project` and `project_assignment`, so their functions run as `SECURITY DEFINER`, owned by `postgres`, with a fixed `search_path`. Without that, an insert into an RLS-protected table would fail and break today's owner and team saves.
- Unique names use `name_key text GENERATED ALWAYS AS (lower(btrim(name))) STORED`.
- Ordered children use a `position int` that runs 1..N, checked by `require_contiguous_positions` (`src/shared/utils/positioning.py`). The UNIQUE constraint is DEFERRABLE INITIALLY DEFERRED, so a reorder can run inside one RPC.

- Status values are Postgres enums created in a guarded `DO $$` block, so the migration is safe to re-run.

Roles

Table	Columns	Constraints and indexes
`project_team_roles`	`name text` (1–100 chars), `name_key`, `position int`, `is_default bool`	UNIQUE (tenant, name_key). UNIQUE (tenant, position) deferrable. UNIQUE (tenant, id), the target of the composite FK from `users`. Defaults seeded by a trigger on `subscriber` insert, plus a one-off backfill. Seed list (MOON-2390), in this order: Project Manager, Product Owner, Business Analyst, Solution Architect, Engineer, QA Lead, Delivery Lead, Executive Sponsor. Seeding skips a name the tenant already has (matched on `name_key`), so the backfill never clashes with roles an admin created.
`project_assignment_team_roles`	`assignment_id uuid` → project_assignment(id) ON DELETE CASCADE, `role_id uuid` → project_team_roles CASCADE	PK (assignment_id, role_id). Index (tenant, role_id). Written only by `save_project_team`. A BEFORE INSERT trigger rejects agent rows (`assigned_user` is null) and roles from another tenant.
`users.default_project_team_role_id` (new column)	`uuid`, nullable. Composite FK `(subscriber, default_project_team_role_id)` → `project_team_roles(tenant, id) ON DELETE SET NULL (default_project_team_role_id)` (Postgres 15 syntax).	No trigger and no endpoint. The composite FK keeps the role in the user's own tenant, and deleting a role clears only this column, not `subscriber`. The frontend writes it with its existing Supabase `users` update; today's `users` RLS already limits that to the user themselves or a tenant admin.

Policies and activities

Table	Columns	Constraints and indexes
client_policies	name, name_key, description text, trigger_type text (catalog key, nullable), trigger_reason text (catalog key, nullable), trigger_params jsonb not null default '{}' (values for the reason's parameters, e.g. a threshold), status (active / archived), revision int, archived_at, archived_by	UNIQUE (tenant, name_key) WHERE status = 'active'. CHECK: description is not blank OR trigger_type is set (meeting 2: a trigger, a description, or both). CHECK: trigger_reason only when trigger_type is set. CHECK: jsonb_typeof(trigger_params) = 'object', and it is '{}' when trigger_type is null; ISS validates its contents. MOON-2389's "Owner and audit metadata" is covered by created_by / updated_by, the revision history and operating_model_audit_trail.
client_policy_revisions	policy_id, revision int, snapshot jsonb (policy plus resolved activities), created_at, created_by	PK (policy_id, revision). Append-only: a trigger rejects UPDATE and DELETE.
client_activities	policy_id uuid nullable → client_policies CASCADE, stage_id uuid nullable → client_methodology_stages CASCADE, name, type text (catalog key), description, agent_id uuid → query_form(form_id), prompt text, is_required bool, position int	CHECK num_nonnulls(policy_id, stage_id) = 1. UNIQUE (policy_id, position) and UNIQUE (stage_id, position), both deferrable. Composite FK (tenant, agent_id) → query_form(tenant, form_id) with ON DELETE SET NULL (agent_id) (Postgres 15 syntax), which keeps tenant intact.
client_activity_roles	activity_id → client_activities CASCADE, role_id → project_team_roles CASCADE	PK (activity_id, role_id). Index (role_id) for role-usage counts.

One shape for every policy. A policy has a trigger, a description, or both; at least one is required, as meeting 2 agreed. The trigger is optional routing metadata that says when to look at the policy; it is not a second kind of policy. The description says what to do and how, and the orchestrator reads it whenever it is present. A policy without a trigger is evaluated by its text alone; a policy without a description is

interpreted from its name, trigger, reason, params and activities. We don't require a catch-all trigger key, and we don't turn descriptions into rules ourselves; interpreting text is the orchestrator's job.

Trigger parameters. The type and reason stay real columns, because the orchestrator and the policy list filter on them and the CHECKs depend on them. Anything a reason needs beyond its name, such as a threshold, a metric or a time window, goes in `trigger_params`. Each (type, reason) pair in the ISS catalog declares a Pydantic model for its parameters; a pair without parameters accepts only `{}`. ISS validates on create and update, and `GET /operating-model/catalog` returns each model as a JSON Schema, so the Create Policy form can render the fields without a frontend release per reason. Each policy revision stores the parameters with the rest of the policy, and a published version references the revision it was built from, so the parameters are pinned like everything else.

Catalog changes are additive: a new reason or a new optional parameter needs nothing else. Renaming or removing a key, or making a parameter required, needs a data migration and the AI team's agreement. We store and serve the parameters; the orchestrator decides when the condition is met.

Not now: several triggers per policy, or AND/OR conditions. Figma shows a single trigger and no story asks for combinations. If that changes, the three columns move one-to-one into a `client_policy_triggers` child table.

Methodologies

Table	Columns	Constraints and indexes
`client_methodologies`	`name`, `name_key`, `type` (one_time / recurring), `status` (draft / active / archived), `revision int`, `current_version_id uuid` nullable, `archived_at`, `archived_by`	UNIQUE (tenant, name_key) WHERE status <> 'archived'. `current_version_id` is a deferrable FK to versions.
`client_methodology_stages`	`methodology_id` → client_methodologies CASCADE, `name`, `description`, `position`	UNIQUE (methodology_id, position) deferrable.
`client_methodology_stage_policies`	`id uuid` (client-generated, the policy item's id), `stage_id` → stages CASCADE, `policy_id` → client_policies ON DELETE RESTRICT, `position` (shared sequence with the stage's direct activities)	PK (id). UNIQUE (stage_id, policy_id). Index (policy_id) for policy usage.
`client_methodology_versions`	`methodology_id` → client_methodologies RESTRICT, `version_number int`, `snapshot jsonb`, `snapshot_hash text`,	UNIQUE (methodology_id, version_number). Append-only.

Table	Columns	Constraints and indexes
	change_note text, published_at, published_by	
client_methodology_documents	methodology_id → CASCADE, name text (original filename), mime text (from the content check, never the client header), size bigint, storage_bucket text, storage_path text, sha256 text, created_at, created_by	UNIQUE (storage_bucket, storage_path). Column names follow ticketing_attachments. The newest row is the current document.

Project runtime

Table	Columns	Constraints and indexes
project_methodology	project uuid PK → project CASCADE, methodology_id RESTRICT, version_id RESTRICT, assigned_at, assigned_by, current_stage_id uuid nullable, current_stage_set_at, current_stage_set_by, revision int	One row per project; no row means no methodology. An RPC checks that current_stage_id exists in the pinned snapshot.

Cross-cutting

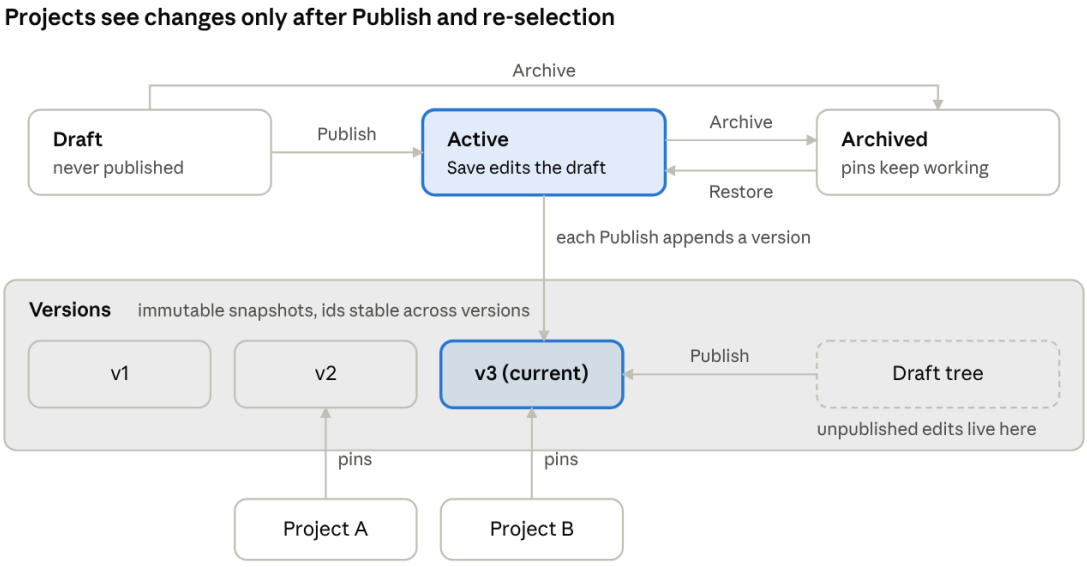
Table	Columns	Constraints and indexes
operating_model_audit_trail	seq bigint identity, txid xid8 DEFAULT pg_current_xact_id(), occurred_at, actor_type (user / ai / system / service), actor_id, entity_type, entity_id, action, project nullable, before jsonb, after jsonb, context jsonb	Indexes (tenant, entity_type, entity_id, seq desc), (tenant, project, seq desc), and (txid, seq) for the feed cursor. Append-only.
audiences	id, tenant, project → project CASCADE, created_at	RLS tenant isolation. Deleted by the daily cleanup once no row targets it.
audience_members	audience → audiences CASCADE, user_id → users CASCADE, via_role_id → project_team_roles SET NULL	PK (audience, user_id). Index (user_id, audience) for the RLS lookup.

Table	Columns	Constraints and indexes
`suggestions` (new columns)	`deduplication_key text`, named as on `feed_items` and `subscriber_events`	UNIQUE (tenant, deduplication_key) WHERE deduplication_key IS NOT NULL, whatever the status, so a replayed event never recreates an applied or dismissed suggestion.
`service_credentials`	`name`, `key_hash` (SHA-256), `scopes text[]`, `allowed_tenants bigint[]` (null = all), `last_used_at`, `revoked_at`. No tenant column: these are platform credentials.	UNIQUE (key_hash). RLS on, no policy. Detail in the AI team: topics to discuss tab.

Methodology lifecycle and versioning

> ***Depends on open question #9.*** *This section describes option C: each project stays on its version until a project admin upgrades it. With option B, Publish moves every project to the new version and there is no upgrade step. With option A there is no draft, Publish or version. Archive depends on open question #8.*

Edits change only the draft. Projects see a change only after **Publish** creates a new immutable version and a project admin selects it.



methodology lifecycle · 3 states, versions, project pins

Project A stays on v2 after v3 is published. It moves only when a project admin re-selects the methodology.

States

Status	Meaning	Can be selected for a project
draft	Created, never published.	No
active	Has at least one published version. Edits after that are an unpublished draft.	Yes; the current version is the default.
archived	Hidden from lists and selection. Projects already pinned keep working.	No

Rules

- **Save** (`PUT …/structure`) works in any status except archived. It bumps `revision`, which is only a concurrency counter (the snapshots are versions), and never touches versions.
    - Ids come from the client. In the structure RPC, an id already in this draft is updated and an unknown id is inserted with that value. An id that belongs to another methodology or tenant fails with 422 `operating_model.methodology.invalid_id`, which reveals nothing about the owner. There are no temporary keys and no id mapping.
- **Publish** creates version N+1, sets `current_version_id` and moves draft to active.
    - It is idempotent: if the new snapshot hash equals the current version's hash, it returns the existing version.
- **Archive** is allowed while projects are pinned. **Restore** returns the methodology to its previous status.
- **Delete** is allowed only while the methodology has never been published; the RESTRICT FK from versions enforces it. Otherwise the API returns 409 `operating_model.methodology.in_use` and the UI offers Archive.
- `has_unpublished_changes` is true when `revision` moved since the last publish, or when any attached policy's live `revision` is newer than the revision in the snapshot.

Publish RPC: `publish_client_methodology(p_tenant, p_methodology_id, p_expected_revision, p_actor, p_note)`

1. Lock the methodology row (`FOR UPDATE`) and compare `revision` with `p_expected_revision`. A mismatch raises `operating_model.methodology.revision_conflict` (409).
2. Validate the tree:
    - at least one stage
    - every activity has a name and a known type
    - roles and agents belong to the tenant
    - attached policies are active
3. Build the snapshot (shape below) and its `snapshot_hash`.
4. Insert into `client_methodology_versions`, then update `current_version_id`, `status` and the published revision.

5. Write a `operating_model.methodology_version.published` row to `operating_model_audit_trail`.

Snapshot shape

The stored snapshot holds the methodology, its stages and direct activities, and **policy references** `{id, revision}`. Policy content stays in the immutable `client_policy_revisions`, so it is never copied into methodology records, as MOON-2391 and MOON-2394 require. Reads (`GET …/versions/{id}` and the internal API) return the resolved view below by joining those revisions, and the result never changes. Ids are stable across versions, so the stage pointer survives an upgrade.

```
{
  "methodology": {"id": "…", "name": "Enterprise Delivery Framework", "type":
"one_time"},
  "version": {"id": "…", "version_number": 3, "published_at":
"2026-11-12T10:00:00Z"},
  "stages": [
    {
      "id": "…", "name": "Initiation", "description": "…",
      "items": [
        {
          "kind": "policy", "id": "…", "policy_id": "…", "revision": 4,
          "name": "New Project Kickoff", "description": "…", "status": "active",
          "trigger": {"type": "project.methodology.stage_changed", "reason": null,
"params": {}},
          "items": [
            {"kind": "activity", "id": "…", "name": "Kick-off meeting", "type":
"scheduling.calendar.create_event", "description": "…", "roles": [{"id": "…",
"name": "PM"}], "agent": {"id": "…", "name": "Emily"}, "prompt": "…",
"is_required": true}
          ]
        },
        {"kind": "activity", "id": "…", "name": "Identify stakeholders", "type":
"insight_content", "roles": [{"id": "…", "name": "BA"}], "agent": null, "prompt":
null, "is_required": false}
      ]
    }
  ]
}
```

Policy revisions

- Every policy save (fields or activities) bumps `client_policies.revision` and appends a row to `client_policy_revisions` inside the same RPC.
- A policy edit never changes a published methodology version. Methodologies that use the policy show "policy updated since last publish" until they are republished.
- Publishing is blocked while a stage references an archived policy (`operating_model.methodology.policy_archived`).

Upgrading a project to a newer version

- A project stays on its pinned version until a project admin re-selects the methodology (open question #18). Project Settings shows "v4 available".

- On upgrade, the RPC keeps `current_stage_id` if that stage still exists in the new snapshot; otherwise it resets to the first stage.
- All of this is recorded as one `project.methodology.version_upgraded` event.
- Type **Recurring** is stored and returned. Decided in #10: a One-Time methodology stops after its last stage; a Recurring one starts again at stage 1, like sprints.

Public API catalog

All endpoints are new routers in integration-sync-service and follow its conventions:

- no `/api/v1` prefix and kebab-case paths
- `page` / `per_page` pagination and `sort=-field`
- errors as `{detail, code, meta, request_id}`
- PATCH bodies use `Unset`
- every route is documented in `postman/collections/`

Module layout follows `src/modules/ticketing/collection_template/` (router, schemas, models, repository, service, error_codes).

Methodology tree format

A stage holds one ordered list, `items`. Every item has `id` and `kind`; the other fields depend on `kind`. Reads and writes use the same envelope, so a new kind later (a checklist, a milestone, a nested policy) is one new variant with no change to stages or to existing kinds. Today there are two kinds, `activity` and `policy`.

Write shape, used by `PUT …/structure` and by policy create and update:

```
type StageItemInput = ActivityItemInput | PolicyItemInput

type ActivityItemInput = {
  kind: "activity"
  id: string              // client-generated UUID, permanent
  name: string
  type: string            // activity type catalog key
  description?: string
  role_ids: string[]
  agent_id?: string
  prompt?: string
  is_required: boolean
}

type PolicyItemInput = {
  kind: "policy"
  id: string              // client-generated UUID of the stage-policy link
  policy_id: string       // the shared policy; its content is not edited here
}
```

Read shape, returned by `GET …/{id}`, `GET …/versions/{version_id}` and the internal context API:

```
type StageItem = ActivityItem | PolicyItem

type ActivityItem = Omit<ActivityItemInput, "role_ids" | "agent_id"> & {
```

```
  roles: { id: string; name: string }[]
  agent: { id: string; name: string } | null
}

type PolicyItem = {
  kind: "policy"
  id: string
  policy_id: string
  revision: number
  name: string
  description: string | null
  trigger: { type: string; reason: string | null; params: object } | null
  status: "active" | "archived"
  items: ActivityItem[]   // read-only inside a methodology
}
```

- **Order** is array order. The RPC writes `position` from the index: one sequence per stage across direct activities and policy links, and one per policy for its own items.
- **Discriminator**: `kind`, as a Pydantic discriminated union (`Field(discriminator="kind")`) in ISS and a TypeScript union in the frontend. An unknown kind returns 422 `operating_model.methodology.unknown_item_kind`.
- **Policies** can be linked once per stage; a second link returns 422 `operating_model.methodology.duplicate_policy`. A policy item carries its own `items`, so the editor draws a policy with the same code as a stage. Policy create and update take `items` with `kind: "activity"` only.
- **Ids** follow the client-generated id rule in the methodology Rules above.

Project Team Roles — module `project_team_role`

Method and path	Guard	Behavior
`GET /project-team-roles`	AuthenticatedUser	Ordered list `{id, name, position, is_default}`.
`POST /project-team-roles`	RequireTenantAdmin	`{name}`. 409 `operating_model.team_role.name_taken`.
`PATCH /project-team-roles/{role_id}`	RequireTenantAdmin	Rename.
`PUT /project-team-roles/order`	RequireTenantAdmin	`{role_ids[]}`, the full ordered list. 422 if any role is missing.
`GET /project-team-roles/{role_id}/usage`	RequireTenantAdmin	`{assignment_count, project_count, activity_count}` for the delete warning.

Method and path	Guard	Behavior
`DELETE /project-team-roles/{role_id}`	RequireTenantAdmin	One RPC deletes the role, its assignments and its activity links. The frontend shows the counts from `GET …/{role_id}/usage` in its confirm modal first; the API does not ask again.
`GET /projects/{project_id}/team`	ViewerProjectSummary	People with an active assignment, each with access level, `granted_via_hierarchy` and roles. The owner appears only when they have an assignment row. Optional `?role_id=` (CSV) keeps members holding any of those roles, which covers MOON-2390's "active project members assigned to a specified role". Active means an active assignment and an active user; agent rows are never returned. 422 `operating_model.team_role.not_found` for a role outside the tenant; a valid role nobody holds returns an empty list. No other filters: the client splits hierarchy members by `granted_via_hierarchy`, and a team is small enough to return whole. One repository method, `find_members(project_id, role_ids=None)`, also backs the internal members endpoint and assignee routing.
`PUT /projects/{project_id}/team`	AdministratorProjectSummary	`{members: [{user_id, access_level, role_ids[]}]}` is the full list of people. Calls `save_project_team` with the caller's JWT, which adds, updates and removes

Method and path	Guard	Behavior
		assignments and sets roles in one transaction. 422 `operating_model.team_role.not_found`; database guard errors (escalation, confidential, limits) come back as 403 or 422 with their message.

Team save RPC: `save_project_team(p_project uuid, p_members jsonb)`. SECURITY INVOKER; ISS calls it through `user.supabase` (the caller's JWT), never the service-role client. All existing `project_assignment` RLS and triggers therefore still apply: `check_user_permissions` and shares, `guard_assignment_escalation`, the confidential guards, `created_by_id` / `last_modified_by`, user tags, hierarchy shares and the sync webhook.

1. Lock the project's people assignments (`FOR UPDATE`).
2. Diff against `p_members`: insert new people, update changed access levels, delete people who are missing. Agent rows and rows with `granted_via_hierarchy` are left alone.
3. Replace each member's rows in `project_assignment_team_roles`.
4. Write one `project.team.changed` event with the before and after member lists.

The owner is treated like anyone else: they get an assignment row only if the payload lists them.

Client Policies — module `client_policy`

Method and path	Guard	Behavior
`GET /client-policies`	AuthenticatedUser	Filters: `search` (name and description), `status`, `usage=standalone|in_methodology`, `trigger_type`. Paged like collection templates: `PaginationParams` (`page`, `per_page`, default 10, max 100) and `PaginationResponse` (`{items, page, per_page, total, total_pages}`), ordered by name, then id. Item: `{id, name, description, trigger, status, updated_at}`, where `trigger` is `{type, reason, params}` or `null`.
`GET /client-policies/recent?limit=3`	AuthenticatedUser	"Recently used" in the Add Policy modal, ordered by the last time each policy was attached to a stage.
`POST /client-policies`	RequireTenantAdmin	`{name, description?, trigger?: {type, reason?, params?} | null, items: ActivityItemInput[]}`, the

Method and path	Guard	Behavior
		same `trigger` object the reads return. Created atomically by an RPC, which also writes revision 1. ISS maps `trigger` to the `trigger_type`, `trigger_reason` and `trigger_params` columns; `null` or no `trigger` clears all three. `params` defaults to `{}` and is validated against the chosen reason's parameter model; a mismatch returns 422 `operating_model.policy.invalid_trigger_params` with the field path, e.g. `trigger.params.value`.
GET `/client-policies/{policy_id}`	AuthenticatedUser	Policy with its activities, `revision` and `used_in`: `[{methodology: {id, name}, stages: [{id, name}], published}]`, one entry per methodology that references the policy (empty when standalone). `published` is true when any published version of that methodology references the policy; an entry can then have `stages: []` after the draft dropped it. The drawer's "Used in N methodologies" warning is the length of `used_in`.
PUT `/client-policies/{policy_id}`	RequireTenantAdmin	Full replace with the same body as `POST`, including `trigger` and activities, plus `expected_revision`. 409 `operating_model.policy.revision_conflict`.
POST `/client-policies/{policy_id}/archive` and `/restore`	RequireTenantAdmin	Archive is allowed while the policy is in use; publishing a methodology that references it is then blocked.
DELETE `/client-policies/{policy_id}`	RequireTenantAdmin	409 `operating_model.policy.in_use` when any stage or published version references it, with the same `used_in` list in `meta`.
GET `/operating-model/catalog`	AuthenticatedUser	The keys defined in ISS code: `{trigger_types: [{key, label, reasons: [{key, label, params_schema}]}],`

Method and path	Guard	Behavior
		`activity_types: [{key, label, suggestion_type}]}`. `params_schema` is the JSON Schema of the reason's `trigger_params`; a reason without parameters gets an object schema with no properties. `suggestion_type` is the suggestion type the activity maps to. The response is the same for every tenant and changes only on deploy, so it is cacheable (`ETag`).

Client Methodologies — module `client_methodology`

Method and path	Guard	Behavior
`GET /client-methodologies`	AuthenticatedUser	Filters: `search`, `status`, `type`, `published` (`true` keeps only methodologies with a published version; the Project Settings methodology select calls `?status=active&published=true&search=`). Paged like the policies list: `PaginationParams` (`page`, `per_page`, default 10, max 100) and `PaginationResponse` (`{items, page, per_page, total, total_pages}`), ordered by name, then id. Item: `{id, name, type, status, stage_names[], current_version, has_unpublished_changes, updated_at}`. `current_version` is `{id, version_number, published_at}` of the latest published version, or `null` before the first Publish. `has_unpublished_changes` is true when the draft was edited after the last Publish or an attached policy has a newer revision than the one that version references. Both feed the status badge (open question #9).
`POST /client-methodologies`	RequireTenantAdmin	JSON `{name, type}`. Creates a draft.

Method and path	Guard	Behavior
PUT `/client-methodologies/{id}/document`	RequireTenantAdmin	Multipart `file`. Attaches a document to an existing methodology; a later upload replaces it. Creating from a document is `POST` then this call. See Document upload.
GET `/client-methodologies/{id}`	AuthenticatedUser	Methodology, `revision`, and the draft tree. Attached policies are expanded read-only.
PATCH `/client-methodologies/{id}`	RequireTenantAdmin	`{name?, type?}` from the Settings modal. Bumps `revision`.
PUT `/client-methodologies/{id}/structure`	RequireTenantAdmin	`{expected_revision, stages: [{id, name, description, items: StageItemInput[]}]}` (see Methodology tree format). Returns `{revision, stages}` in the read shape.
POST `/client-methodologies/{id}/publish`	RequireTenantAdmin	`{expected_revision, change_note?}`. Returns the version.
GET `/client-methodologies/{id}/versions` and `/versions/{version_id}`	AuthenticatedUser	Version list (with pagination), or one snapshot.
POST `/client-methodologies/{id}/archive`, `/restore`	RequireTenantAdmin	Archive removes the methodology from project selection; projects already pinned keep their version. Restore makes it selectable again. There is no Duplicate: a new version of the same methodology is the way to change it.
DELETE `/client-methodologies/{id}`	RequireTenantAdmin	409 `operating_model.methodology.in_use` once it has been published. Use Archive instead.
GET `/client-methodologies/{id}/document`	AuthenticatedUser	307 redirect to a signed URL.

Project runtime — module `project_methodology`

Method and path	Guard	Behavior
`GET /projects/{project_id}/methodology`	ViewerProjectSummary	`{methodology {id, name, type}, version {id, version_number, published_at}, upgrade_available {id, version_number} | null, current_stage {id, name, position, total, set_at, set_by {id, name}} | null, assigned_at, assigned_by {id, name}, revision}`. Body is `null` (200) when the project has no methodology. `upgrade_available` is set only when a newer published version exists and drives the "Upgrade" notice. `revision` is sent back on both `PUT`s. Not embedded in `GET /projects/{project_id}`, which stays the thin `ProjectSummary`.
`PUT /projects/{project_id}/methodology`	AdministratorProjectSummary	`{methodology_id, version_id?, revision?}`; the default is the current version, and `revision` is required once the project has a methodology. One endpoint covers assign, replace and upgrade. Returns 422 `operating_model.project_methodology.not_assignable` when the methodology is not active or has no published version.
`DELETE /projects/{project_id}/methodology`	AdministratorProjectSummary	Removes the association. History and versions remain.
`PUT /projects/{project_id}/methodology/stage`	AdministratorProjectSummary	`{stage_id, note?, revision}`. Any stage in the pinned version, forward or back. Nothing checks earlier stages'

Method and path	Guard	Behavior
		activities; that check comes with activity tracking (Future phase).

Permissions

Every check reuses an existing guard, and no new permission model is needed for the first release.

- **Org-level writes** use `RequireTenantAdmin`, which checks `users.company_admin` (`src/modules/auth/guards.py`).
- **Project-level checks** use the access-level dependencies in `src/modules/project/deps.py`. Company admins already resolve to level 400 on every project.
- **Project Team Roles never change access.** Role tables are not read by `get_project_user_access_level` or by any RLS policy.

Action	Tenant admin	Project admin (400)	Project editor (300)	Project member (100–200)	AI service
Read the role, policy, methodology and catalog libraries	Yes	Yes	Yes	Yes	Yes (internal API)
Create, edit, reorder, delete roles	Yes	No	No	No	No
Create, edit, archive, delete policies	Yes	No	No	No	No
Create, edit, publish, archive, delete methodologies	Yes	No	No	No	No
Select, replace, upgrade, remove the project methodology	Yes	Yes	No	No	No
Assign Project Team Roles to members	Yes	Yes	No	No	No
Move the current stage	Yes	Yes	No	No	Through `change_stage` suggestions only, applied by a project admin
View project methodology and stage	Yes	Yes	Yes	Yes	Yes
Set own default project role	Yes	Yes	Yes	Yes	No
Create suggestions	No	No	No	No	Yes (internal API)

Action	Tenant admin	Project admin (400)	Project editor (300)	Project member (100–200)	AI service
Apply or dismiss a suggestion	Yes	Yes	Yes	Only when assigned (open question #24)	No

Why these levels

- **Team saves need Administrator (400), the same as today.** Adding a member or changing an access level already passes through the `guard_assignment_escalation` trigger, which requires `has_preexisting_access(project, tenant, 400)`: a direct active assignment at 400, the project owner or a company admin. Roles follow the same bar, because MOON-2390 says users who cannot manage project team assignments cannot change Project Team Role assignments either. ISS checks `AdministratorProjectSummary` first, so an Editor gets a clean 403 instead of a database error. Someone who reaches 400 only through a share or the role hierarchy passes ISS but is still refused by the trigger, as today; the RPC maps that error, and the assignment-limit error, to 403 and 422 with their message.
- **Methodology selection uses Administrator (400)** because the frontend only opens Project Settings at level 400. The existing suggestion settings use the stricter `RequireProjectOwnerOrAdmin`, which rejects an assignee at level 400; open question #17 confirmed level 400 for methodology selection.
- **The AI never writes project state directly.** base.md asks for human-in-the-loop, so the orchestrator proposes stage moves as `change_stage` suggestions. Direct writes (autonomy) are a future phase.

Audit and events

> ***Possibly in scope.*** *This section needs refinement, and we may not build it in the first version. The stories (MOON-2393, MOON-2394) ask for audit history, so leaving it out needs Product's agreement.*

Every write RPC appends to `operating_model_audit_trail` in the same transaction. That one table is the audit log required by MOON-2389 and the outbox behind the AI event feed.

Why not `tenant_audit_trail`

- Its trigger (`private.log_tenant_audit_trail`) returns early unless `auth.role() = 'authenticated'`. ISS writes with the service-role client, so nothing would be logged.
- It is off unless the tenant enables `audit_logging_enabled`.
- The feed view prunes rows after 30–180 days.

How rows are written

- One SQL helper does the insert: `private.log_operating_model_audit(p_tenant, p_actor_type, p_actor_id, p_entity_type, p_entity_id, p_action, p_project, p_before, p_after, p_context)`.
- Every RPC takes `p_actor_type` and `p_actor_id` explicitly, because `auth.uid()` is null under the service role.
- `before` and `after` hold only the changed fields; inserts and deletes hold the whole row.
- `context` carries `request_id`, `source` (the channel: `ui` / `internal_api` / `migration`; who acted is `actor_type`), and related ids (`methodology_id`, `policy_id`, `suggestion_id`).
- `project.project_owner` is still written directly by the frontend. A new `AFTER UPDATE OF project_owner` trigger writes `project.owner.changed` (actor = `last_modified_by`) so owner changes reach the event feed.

Event catalog

Event, suggestion source and scope keys use one style, the one ISS already has: `<domain>.<entity>.<event in past tense>`, as `callpilot.call.ended`, `project.health.status_changed` and `ticketing.ticket.user_mentioned` do. Project-level entities use the `project.` domain; the org library (roles, policies, methodologies) uses `operating_model.`. A policy trigger key is the event key where one exists, e.g. `project.methodology.stage_changed`.

Module layout and error codes. All new ISS code lives in one package, `src/modules/operating_model/`, with submodules `team_role`, `policy`, `activity`, `methodology`, `project_methodology` and `orchestration` (the internal router), as `ticketing/` does. Error codes follow the module path the way ISS already does (`ticketing.collection.not_found`, `suggestion.apply.failed`): `operating_model.<submodule>.<code>`, e.g. `operating_model.methodology.in_use`, `operating_model.team_role.not_found`, `operating_model.project_methodology.not_assignable`. Each submodule has its own `error_codes.py`.

Entity type (event = entity type + "." + action)	Actions	In AI feed
`operating_model.team_role`	created, renamed, reordered, deleted	Yes
`project.team`	changed (members added, removed, access level or roles changed)	Yes
`project.owner`	changed	Yes
`operating_model.policy`	created, updated (with revision), archived, restored, deleted	Yes
`operating_model.methodology`	created, settings_updated, structure_saved, archived, restored, deleted	Only archived and deleted; drafts never reach runtime.

Entity type (event = entity type + "." + action)	Actions	In AI feed
`operating_model.methodology_version`	published	Yes
`operating_model.methodology_document`	uploaded	Yes; starts extraction.
`project.methodology`	assigned, replaced, version_upgraded, removed, stage_changed. A stage change made by applying a suggestion carries `suggestion_id` in its context.	Yes
`project.health`	status_changed, mirrored by a trigger on project_health_webhook_events	Yes. One feed for the orchestrator instead of two.

Feed semantics

- **Ordering and cursor**: the feed is ordered by `(txid, seq)` and only returns rows whose `txid < pg_snapshot_xmin(pg_current_snapshot())`.
    - Every transaction below that horizon has finished, so no row can appear later behind the cursor, however long a transaction runs.
    - The cursor is the last `(txid, seq)`, encoded as an opaque string.
- **Filtering**: the feed filters by the key's `allowed_tenants` and leaves out confidential projects unless that is decided otherwise.
- **Delivery**: consumers must be idempotent on `seq`; delivery is at-least-once.
- **Retention**: there is no pruning. Expected volume is a few hundred rows per tenant per day.

Where people see history

- Project, methodology and policy: no history view or endpoint in the first version. No story or Figma frame shows history; the stories only require that changes are recorded, which the RPCs do. A history view reads the same table later (Future phase).
- The org Audit Logs page (`tenant-audit.service.ts`) reads `tenant_audit_trail_feed`. Showing these events there needs a UNION in that view; that will be decided later.

Project runtime: current stage

> ***Possibly in scope.*** *This section needs refinement, and we may not build it in the first version. No story asks for it: MOON-2389 needs no visual stage tracker in the first release (`base.md:61`), and MOON-2393 excludes stage progress in the project workspace (`:387`). Only the Figma stage card shows it.*

A project with a methodology keeps one piece of runtime state: its current stage. It drives the Overview stage card ("Stage 2/5 · Discovery and Scope"). Activity tracking, the "Medium adherence" label, deviations and manual marking are a future phase.

Current stage

- Assigning a methodology sets `current_stage_id` to the first stage.
- A person changes it from the stage card's edit button with `PUT …/methodology/stage`.
- The orchestrator does not move stages by default. It creates a `project.methodology.change_stage` suggestion with evidence, for example the call where scope sign-off was agreed. Applying it runs the same RPC a person uses, so only a project admin (400+) or tenant admin can apply it.
- The card shows the stage name only. Figma adds the next open activity after a colon ("Gap analysis"); that needs activity tracking, which is a future phase.
- A project with no methodology keeps showing the Certinia stage card, as today. When both exist, which card shows is an open question.

Visibility standard

Suggestions, feed items and subscriber events already share one pattern for who sees what, but it was never written down, and suggestions use only half of it. Every user-facing row follows these rules from now on, starting with MOON-2389.

Concept	Columns	Meaning	Enforced by
Target	`target_type`, `target_id`	**Who** may see the row	RLS only
Scopes	`*_scopes(entity_type, entity_id)`	**Where** it shows (list filters)	Nothing; never a permission
Subject (feed only)	`subject_type`, `subject_id`	**What** it is about	Read-time resolution

Rules

1. Only the target decides who sees a row. A feature never adds its own visibility table or RLS clause.
2. There are three target types: `project` (everyone with access to the project, as today), `user` (one person; feed mentions already use it) and the new `audience` (a stored list of people inside one project).
3. A row derived from another row copies its target. The feed item for a suggestion takes the suggestion's `target_type` and `target_id`, so the two never disagree; `SuggestionCreatedActivity` stops hardcoding `project`.
4. Producers state an intent, and one ISS `AudienceResolver` turns it into a target. The intent is `project`, `user_ids`, `role_ids` or a mix. Roles resolve to the people holding them at that moment. An intent that resolves to nobody falls back to the project owner.
5. Seeing and acting are separate checks. The target only controls who sees a row; each module decides who may act on it. For suggestions, an assignee may apply at any access level (proposed,

open question #24) and anyone else needs Editor; a type that changes protected project state sets its own minimum (`change_stage` needs project admin).

Audience tables (specs in Domain model → Cross-cutting)

- `audiences(id, tenant, project, created_at)`: one audience per row in v1, never shared between rows.
- `audience_members(audience, user_id, via_role_id)`: `via_role_id` records why a person is included ("assigned as BA"); null means they were named directly. A person is in an audience once; if they hold several targeted roles, `via_role_id` is the first of them in the order the orchestrator listed them in `targets`.
- A trigger on `project_assignment` delete removes that user's memberships in the project's audiences. If that empties an audience, its rows move to the project owner.

RLS

The `audience` branch is a plain SQL policy on `suggestions` and `feed_items`, not a new case in `check_entity_access`. That function is plpgsql and runs once per row; this subquery runs once per query and is matched as a hashed set.

```
CREATE POLICY "Read access via audience" ON public.suggestions FOR SELECT USING (
  target_type = 'audience'
  AND target_id::uuid IN (
    SELECT audience FROM public.audience_members WHERE user_id = (SELECT auth.uid())
  )
);
```

- Policies are permissive and OR together, and `check_entity_access` returns false for `audience`. Rows targeted at `project` or `user` cost the same as today.
- Membership is the only check. The removal trigger keeps it in line with project access.

Cleanup

- `target_id` is polymorphic text, so no foreign key can delete an audience when its rows go.
- A daily step next to `cleanup_suggestions` and `cleanup_expired_feed_items` in workflow-service deletes audiences older than one day that no suggestion, feed item or subscriber event targets. Members cascade.
- `suggestions`, `feed_items` and `subscriber_events` each get a partial index `(target_id) WHERE target_type = 'audience'` so the check stays cheap.
- An audience lives as long as anything points to it, including applied and dismissed suggestions, so history stays visible to its assignees.

Code changes

- `SuggestionTargetType` and `FeedTargetType` gain `audience`.
- The new `AudienceResolver` in ISS is used by suggestion ingestion.
- `SuggestionCreatedActivity` copies the suggestion's target.
- Existing rows don't change; `project` and `user` targets stay valid.

Later steps are in the Future phase tab.

Suggestion integration

Orchestrator suggestions go into the existing store in `src/modules/suggestion/`, so listing, apply, dismiss, prioritization and the feed keep working. Five changes are needed.

What exists today

- suggestion-service only generates. ISS stores, lists and applies (`services/manager_service.py`).
- `target_type` is always `project`. Anyone with view access sees a suggestion, and applying needs Editor.
- `type` is a closed enum (`enums/base.py`). Each type has a resolver and an executor under `types/`.
- `trigger` jsonb and `suggestion_scopes` (generic `entity_type` / `entity_id`) describe where a suggestion came from.
- Ingestion is `POST /suggestions/bulk/create` behind the shared API key.

1. Assignees

- Assignees are an `audience` target (see Visibility standard): the suggestion gets `target_type = 'audience'`, and each assignee is an `audience_members` row.
- One suggestion can have several assignees, and the first one to apply finishes it for everyone. This avoids two people sending the same email.
- The orchestrator sends `targets`, a list of `{kind, id}` objects where `kind` is `project_role` or `user` today and more kinds (for example `team`) can be added later. The `AudienceResolver` resolves project roles to the users holding them on that project at ingestion time and stores `via_role_id`.
- An unresolvable role (nobody holds it) falls back to the project owner, with `via_role_id` kept so the UI can say "No BA assigned".
- **Visibility**:
    - assignees see it
    - nobody else sees suggestions that have assignees, project admins included; suggestions without assignees stay visible to the whole project, as today.

  `GET /suggestions` gains `assigned_to_me=true`, and RLS gains the audience policy. The suggestion's feed item copies its target, so only the assignees see it in the feed too.
- **Apply**: an assignee may apply or dismiss their own suggestion at any access level, Viewer (100) included; non-assignees still need Editor (300). Exception: `project.methodology.change_stage` always needs project admin (400) or tenant admin, assignee or not, because its endpoint does. The assignee rule is open question #24.

2. Source attribution

- New `suggestion_scopes` entity types: `operating_model.activity`, `operating_model.stage`, `operating_model.policy`. The existing PK (suggestion, entity_type) allows one of each, which is enough.
- New `SuggestionTriggerType` values: `orchestrator.methodology`, `orchestrator.policy`, `orchestrator.event`.

- `trigger` jsonb gains `methodology_version_id`, `policy_revision` and the event `seq` that caused it.
- `SuggestionSourceInfo` gets resolvers for the new sources, so the card can show the "Methodology" badge and "Policy: Low Project Health".

3. Deduplication and lifetime

- `RemovePreviousCallSuggestions` in workflow-service must also filter on `trigger->>'type' = 'callpilot.call.ended'`. Otherwise regenerating a call deletes orchestrator suggestions that carry the same call scope. The trigger type already records who created a suggestion, so no new column is needed.
- New column `deduplication_key`, named as on `feed_items` and `subscriber_events`, unique per tenant whatever the status. A replayed event returns the existing id and never recreates a suggestion someone already applied or dismissed.
- `expires_at` is honored as today.
- Quota: only workflow-service's call pipeline calls `increment_suggestion_generation_usage`, once per generation run. ISS ingestion never does, so orchestrator suggestions are unmetered unless we add a counter.

4. Stage move suggestion

One small typed suggestion lets the orchestrator propose a stage move without changing state itself:

- `project.methodology.change_stage` with payload `{stage_id, evidence}`. Its executor checks that the person applying is a project admin (400+) or tenant admin, and returns `suggestion.apply.insufficient_access` otherwise; anyone else can still see and dismiss it. The orchestrator targets it at the project owner and admins (AI team: topics to discuss).

Its executor calls the same RPC a person uses.

5. Activity type to suggestion type

The Figma activity types already match the executable types, so most activities need no new executor. Insight / content creation and Generate document match none; the orchestrator does not suggest them in v1, so they stay pending until Product defines what they should do.

Activity type (Figma)	Catalog key	Suggestion type	Executed by
Insight/content creation	`insight_content`	None in v1	Not suggested
Email	`email`	`messaging.email.send_email`	ISS email executor
Calendar	`calendar`	`scheduling.calendar.create_event`	ISS calendar executor
Slack/Teams	`slack_teams`	`messaging.slack.post_update` or	ISS Slack / Teams executors

Activity type (Figma)	Catalog key	Suggestion type	Executed by
		`messaging.teams.post_update`, depending on the tenant's integration	
PM Tool/Jira	`pm_tool`	`project.ticketing.create_ticket` or `update_ticket`	Native or Jira ticketing
Generate document	`generate_document`	None in v1	Not suggested
Certinia suggestion A	`certinia_stage`	`project.certinia.update_stage`	Salesforce, through the user's MCP connection
Certinia suggestion B	`certinia_raid_create`	`project.certinia.create_raid_record`	Same
Certinia suggestion C	`certinia_raid_update`	`project.certinia.update_raid_record`	Same

The Certinia rows are a guess, because the Figma type list only says "A / B / C".

Document upload

Uploading stores the document with the methodology and emits `operating_model.methodology_document.uploaded`; nothing else. Whether and when the AI team extracts stages and activities from it is open question #14; turning extracted content into methodology changes is a future phase (see the Future phase tab).

Upload: `PUT /client-methodologies/{id}/document`

1. Check that the methodology exists in the tenant and is not archived, and that exactly one file was sent.
2. Check the size against `METHODOLOGY_DOCUMENT_MAX_BYTES` (25 MB proposed, the same as ticket attachments). Re-check the real byte count after reading, as `ticket_import` does.
3. Detect the type from the content, never from the name or the client MIME type:
    - PDF starts with `%PDF-`.
    - DOCX and XLSX are ZIP files. Open them with stdlib `zipfile` and require `word/document.xml` or `xl/workbook.xml`.
    - Reject macro-enabled content types and encrypted files.
    - The extension must match the detected type.

    No new dependency is needed.
4. Upload with `FileStorage.upload` (`src/modules/file/storage.py`) to a private bucket `operating-model`, at `{tenant}/methodologies/{uuid}/{sanitized_filename}`. The uuid keeps keys unique. Buckets are created outside migrations, as `project-relay` was.

5. Call RPC `attach_client_methodology_document`. It inserts the `client_methodology_documents` row and writes the `operating_model.methodology_document.uploaded` event. Earlier rows stay as history; the newest row is the current document.
6. If the RPC fails, call `FileStorage.delete_many([path])`, log the orphan path if that fails too, and return the error. The methodology keeps its previous document, if any, and the client can retry against the same `id`.

Error code	HTTP	When
`operating_model.methodology.file_missing`	422	No file or more than one file
`operating_model.methodology.file_too_large`	422	Over the size limit
`operating_model.methodology.file_type_unsupported`	422	Not PDF, DOCX or XLSX by content
`operating_model.methodology.file_type_mismatch`	422	The extension does not match the content
`operating_model.methodology.file_unreadable`	422	Corrupt, encrypted or macro-enabled
`operating_model.methodology.storage_failed`	502	Storage error; the message is generic and leaks no paths

Download: `GET …/document` redirects (307) to a signed URL that lives 10 minutes, with the original filename in the download header.

Tenancy, concurrency, performance

The database itself enforces tenant boundaries, so a bug in one RPC cannot link one tenant's role to another tenant's activity.

Tenancy

- Every table, children included, carries `tenant`. FKs point at `subscriber(tenant)` (94 existing tables do; 19 use `subscriber(id)`).
- Cross-entity references use **composite FKs** on `(tenant, id)`, backed by `UNIQUE (tenant, id)` on the parent. Examples: activity → role, stage → policy, activity → agent.
    - The agent case needs a new `UNIQUE (tenant, form_id)` on `query_form`. `form_id` is already unique, so the constraint always holds.
    - Nullable references use `ON DELETE SET NULL (column)`, so the delete never nulls `tenant`.
- RPCs take `p_tenant` from `user.tenant`, or from the project row on project routes. A request body never supplies the tenant.
- A reference to another tenant's id returns 404, not 403, so ids do not leak.
- The internal API derives the tenant from the project or methodology it loads. Tenant-wide routes carry the tenant in the path and check it against the key's `allowed_tenants`.

- **Confidential projects**: ISS checks access with `get_project_user_access_level`, which lacks the confidential check that `get_project_if_access_level` applies. The fix must ship before runtime data is exposed. Until decided otherwise, the event feed leaves confidential projects out (see Feed semantics).

Concurrency

- Optimistic revisions: `client_methodologies.revision`, `client_policies.revision`, `project_methodology.revision`. A stale write returns 409 `*.revision_conflict` with the current revision in `meta`.
- Each RPC locks its aggregate root `FOR UPDATE` first: the methodology, the policy, or the project_methodology row.
- Reordering runs in one RPC, using deferrable unique constraints on `position`.
- Suggestion apply keeps its existing distributed lock. The first assignee to apply wins.

Limits

Item	Limit	Error code
Name	200 characters	`*.name_too_long`
Description	5,000 characters	`*.description_too_long`
Prompt	10,000 characters	`operating_model.activity.prompt_too_long`
Stages per methodology	50	`operating_model.methodology.too_many_stages`
Activities per stage or policy	100	`operating_model.activity.too_many`
Roles per activity	20	`operating_model.activity.too_many_roles`
Document size	25 MB	`operating_model.methodology.file_too_large`

Performance

- Expected size per tenant: up to 50 methodologies, 20 stages each and 30 activities per container, so one tree is under about 1,000 rows. A snapshot is under 200 KB.
- The draft tree is read by one RPC that returns jsonb. That avoids an N+1 pattern of PostgREST calls from Python.
- The context API caches in Redis per project for 60 seconds. Events on that project (stage, roles, methodology) invalidate the entry.
- Production runs Postgres 15.8, so migrations avoid newer features (the `combine-sprint-migrations` skill has the grep list). `ON DELETE SET NULL (column)` and `xid8` are both available in 15.

Testing strategy

Most of the logic lives in plpgsql RPCs, so the integration tier carries the weight. ISS already runs it against real Postgres, PostgREST and GoTrue (`tests/harness/`).

Tier	Location	Covers
Unit	`integration-sync-service/tests/unit/modules/<module>/`	Request validators and limits, the catalog mapping, file-type detection, role-to-user resolution, error-code mapping.
Integration	`integration-sync-service/tests/integration/modules/<module>/`	Every endpoint for admin vs member vs outsider (401 / 403 / 404). Tenant isolation with two factory tenants. Atomicity: a failure midway persists nothing. Revision conflicts. Role delete cascades. Publish idempotency. A pinned project is unaffected by a later publish. Suggestion ingestion with role resolution and deduplication. One event row per action. Feed order and safety lag.
Migration	`combine-sprint-migrations` skill (`roundtrip.sh`)	up → down → up on a scratch database, parity with `schema.sql`, Postgres 15 compatibility.
Cross-service	`workflow-service/tests`	`RemovePreviousCallSuggestions` leaves orchestrator suggestions alone.
Contract	Shared JSON Schema exported from the Pydantic models	The AI team validates its payloads against the schemas and example fixtures in CI. We run the same fixtures through our ingestion endpoint.
Frontend unit	Jest, specs next to components	NGXS states, the canvas store (undo/redo, tree serialization), form validation.
Frontend e2e	Playwright (`frontend/tests/`)	Create a role and assign it in the team modal. Create a policy with activities. Build, save and publish a methodology. Assign it in Project Settings. Move the stage from the stage card.

Two setup notes:

- The test harness loads `schema.sql`, so a new table is visible to tests only after `make pull-schema` (or a harness SQL file) includes it.
- Each route change also updates its Postman YAML under `postman/collections/`.

Risks

Risk	Impact	Mitigation
The shared `X-API-KEY` is reused for the AI team.	Cross-tenant read and write with all flags on.	The scoped key must exist before any access is handed out.
Call regeneration deletes orchestrator suggestions.	Suggestions disappear silently.	The trigger-type filter in `RemovePreviousCallSuggestions` ships with the ingestion endpoint.
The closed suggestion-type enum blocks new activity types.	Every new type costs work in ISS and the frontend.	Map onto existing types. Add new types only when product asks for a specific UI.
"Stage" means three things.	UI confusion and wrong data shown.	Use "methodology stage" in code and API, and get a product decision on the Overview card (open question #19).
No Python transactions in ISS.	Partial writes if logic leaks into Python.	Every multi-row write is one RPC, and code review enforces it.
Frontend writes to `project`, agent assignments and `users.default_project_team_role_id` bypass ISS.	No event for owner or member changes.	DB triggers write events for owner changes. People on the team go through ISS, and roles cascade with the assignment row.

Sources

Read for this doc on 2026-10-26. Paths are relative to the `veda` workspace.

Product

- `features/client-methodology/base.md`: MOON-2389 to MOON-2394.
- `features/client-methodology/meetings/1.md`: roles, Gate → Stage, Action → Activity, AI team split.
- `features/client-methodology/meetings/2.md` (20 Oct): policies, triggers.
- Figma — Company Methodologies. Frames: Org Defaults tiles, Company Methodologies list, Create Methodology, Upload tooltip, Methodology editor, Editor + Engage with Agent, Methodology Settings, Create Stage, Create Activity, Add Policy, Policies list, Create Policy, Create Policy – Add Activity, Trigger dropdown, Type dropdown, Project Interface – Project Roles, Account Settings – Project role, Project Overview (Certinia stage), View methodology suggestion, Manage project team, Project Settings – General, Project Settings – Methodology, Method Deviation.

integration-sync-service

- `schema.sql`: `project`, `project_assignment`, `suggestions`, `suggestion_settings`, `suggestion_scopes`, `tenant_audit_trail`, `private.log_tenant_audit_trail`, `project_health_webhook_*`, `users`, `query_form`.
- `src/modules/ticketing/collection_template/`, the reference module.
- `src/modules/suggestion/`: routers, `services/manager_service.py`, `enums/base.py`, `types/`.
- `src/modules/project/deps.py`, `src/modules/auth/guards.py`, `src/shared/deps.py`.
- `src/modules/file/storage.py`, `src/modules/ticketing/attachment/_guards.py`.
- `src/modules/ticketing/ticket_event/`, the precedent for an event history.

Other services

- suggestion-service: `src/modules/api/`, `src/modules/engine/ARCHITECTURE.md`, `CONTEXT.md`.
- workflow-service: `src/workflows/generate_call_suggestions.py`, `src/event_handlers/`, `src/modules/engine/`.
- frontend: `org-defaults/pages/org-list/`, `project-config-list/`, `users-actions-modal/`, `settings-modal/`, `certinia-stage-card/`, `agent-studio/.../agent-workflow/`, `suggestion-card/`.
- pathfinder-service: `main.py` `/textextract`, `mimetypes_list.py`.

AI team: topics to discuss

> 🚧 ***A proposal to discuss, not an agreement.*** *We don't know your architecture yet. Each topic below is one decision we need to make together: the question, the options, what we propose, and what we need from you. Where your design differs, we change ours.*

New to the feature? The Overview tab explains it in plain words.

In short

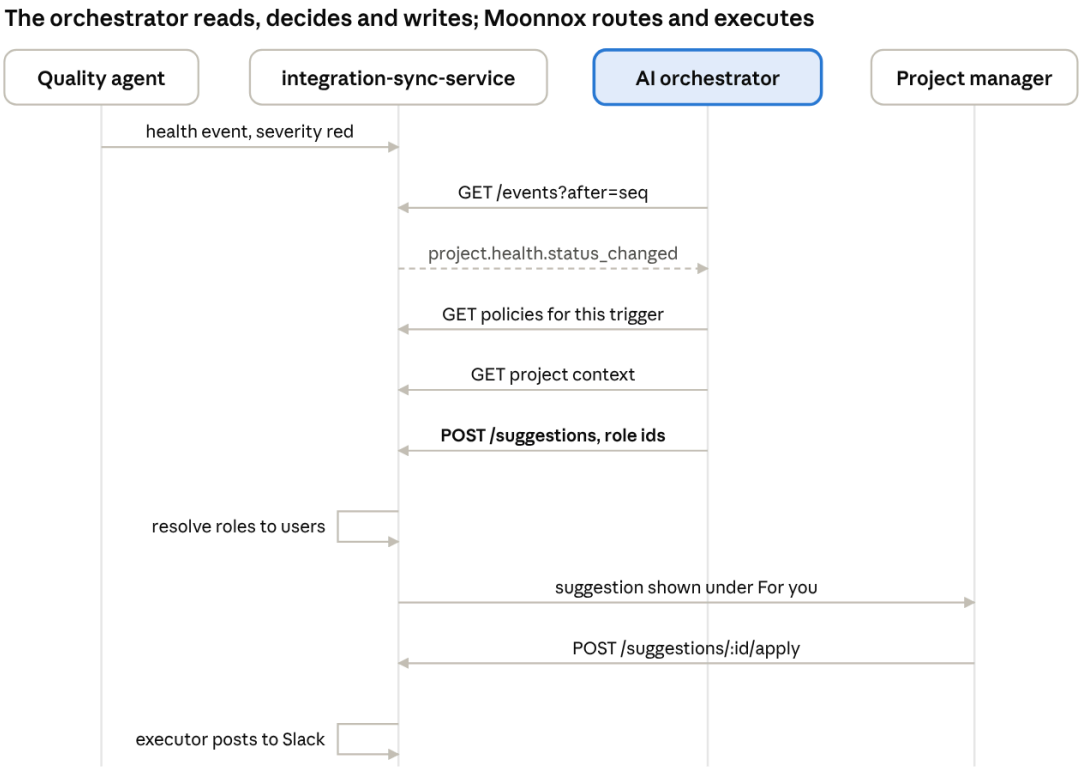
- **Veda stores the client's way of working:** team roles, policies, and methodologies made of stages and activities. Admins manage them in the UI.
- **You decide what should happen and when,** using that data plus the signals you already have (calls, project health, chat).
- **You send us suggestions.** Whether they are addressed to roles or to specific people is still open (topic 4). Veda shows them to the right people and runs them when someone applies them.
- **People stay in control.** In the first version the orchestrator never changes a project directly: it suggests, a person applies.

Topics at a glance

#	Topic	What we propose	What we need from you
1	How the orchestrator learns that something happened	You read an event feed from us (`GET /events?after={cursor}`); a webhook or your own channel also work	Which option fits how the orchestrator is built
2	What it reads from Veda	Read endpoints for the project context, methodology versions, a client's policies and the catalog	Anything missing to decide what to suggest
3	How suggestions come back	`POST /projects/{project_id}/suggestions` with a type, payload, `targets`, a deduplication key and the source; the activity type picks the suggestion type	Whether this shape works
4	Who receives a suggestion	Not decided: you send roles and Veda finds the people, or you send the people	Which option fits, and whether you ever need a specific person
5	Shared lists and exact keys	One catalog in Veda (trigger types and reasons, activity, suggestion and event types, and more), read with `GET /internal/orchestration/catalog`	Confirm the lists and names, or tell us what differs

#	Topic	What we propose	What we need from you
6	Stage changes	A project admin moves the stage; you can suggest the move with `project.methodology.change_stage`	Whether this works, and whether you detect stage changes yourselves

The whole loop in one example



flow A · health turns red to an applied Slack update

1. Project health turns red. Veda adds a `project.health.status_changed` event to the feed.
2. You read the event, look up the client's policies for that trigger (here "Low Project Health"), and load the project's context: the team with their roles, the methodology and the current stage.
3. You send one suggestion per activity in the policy, such as "Post a Slack update", for the PM.
4. Veda shows the suggestion to the project's PM.
5. The PM applies it, and Veda posts the message to Slack.

Methodologies work the same way: when a project enters a stage, you get `project.methodology.stage_changed` and suggest that stage's activities.

This is how we understand the flow, but we're not sure it matches your expectations. Correct us wherever it works differently on your side.

Topics to decide

In the order we'd like to discuss them. Topics 1 to 5 block our build; topic 6 can follow. The full tracker, with owners, is in Open questions.

1. How does the orchestrator learn that something happened?

The orchestrator needs to know when something happens, so everything starts with an event. Example: on the Coca Cola project, the call where the client signed off the scope has just ended, and an hour later project health dropped from green to red. The orchestrator has to find out about both before it can suggest anything.

We see three ways to do it:

- **You read a feed from us (our proposal).** Every few seconds you ask "what happened since my last read?" with `GET /events?after={cursor}`, then save where you stopped. You host nothing, and if either side is down for a while you simply continue from the saved point.
- **We push events to you (webhook).** We call a URL you give us whenever something happens. It's faster, but we need retries on our side and you need a public endpoint.
- **You already have something in mind.** Tell us what it is and we will fit into it.

Each event says what happened and to which records (by id), not the full data. Some example events:

Event	When it happens
`project.methodology.stage_changed`	A project moved to another stage
`project.methodology.assigned` / `replaced` / `removed`	A project got, changed or lost its methodology
`project.health.status_changed`	A project's health changed (an existing signal)
`project.team.changed`	People or roles on a project changed
`operating_model.policy.created` / `updated` / `archived` / `deleted`	A policy was created, updated, archived or deleted
`operating_model.methodology_version.published`	A new methodology version exists

We need from you: which of the three options fits how the orchestrator is built.

2. What does the orchestrator need to read from Veda?

Once you know health turned red on Coca Cola, you need the situation to decide what to suggest: who is on the team and in which role, which methodology the project follows, which stage it is in, and which of the client's policies apply.

We'd offer a few read endpoints under `/internal/orchestration`:

Endpoint	What it gives you
`GET /projects/{project_id}/context`	One project: the team with roles, its methodology version and current stage, the policies that apply
`GET /methodology-versions/{version_id}`	A whole methodology version: stages, activities and policies
`GET /tenants/{tenant}/policies?trigger_type=`	A client's active policies with their activities
`GET /catalog`	The shared lists and their exact keys, the same for every client (topic 5). A client's own project team roles come from the project context.

For Coca Cola, the context would look like this (shortened):

```
{
  "project": {"id": "…", "name": "Coca Cola", "status": "Pre-Sale"},
  "team": [
    {"user_id": "…", "name": "Anna Smith", "access_level": 400, "is_owner": true,
     "roles": [{"id": "…", "name": "PM"}]}
  ],
  "methodology": {
    "id": "…", "name": "Enterprise Delivery Framework",
    "version": {"id": "…", "version_number": 3},
    "current_stage": {"id": "…", "name": "Discovery and Scope", "position": 2,
"total": 5}
  },
  "policies": {"attached_to_current_stage": ["…"], "standalone": ["…"]}
}
```

We need from you: is anything missing that you need to decide what to suggest?

3. How does the orchestrator send suggestions back?

The orchestrator never changes a project itself. It proposes an action, and a person applies or dismisses it. Example: the client's "Low Project Health" policy says to post a status update to Slack, so you send a "Post Slack update" suggestion for Coca Cola, and the PM applies it.

You'd send suggestions with `POST /projects/{project_id}/suggestions`, under the same `/internal/orchestration` prefix. The project is in the path, as in topic 2, so one request carries suggestions for one project, and we check that your key may see that project before reading the body:

```
{
  "suggestions": [{
    "type": "messaging.slack.post_update",
    "title": "Post Slack update",
    "description": "Project health turned red; share a status update with the team.",
    "payload": {"message": "…", "type": "internal"},
    "targets": [{"kind": "project_role", "id": "<PM role id>"}],
    "deduplication_key": "policy:…:project:…:event:981:activity:…",
    "source": {"kind": "operating_model.policy", "policy_id": "…", "activity_id":
"…", "event_seq": 981}
```

```
  }]
}
```

`targets` is a list of `{kind, id}` objects, so the shape stays the same whatever we decide in topic 4. A target can be a project team role (`{"kind": "project_role", "id": "…"}`) or a person (`{"kind": "user", "id": "…"}`). New kinds, such as a team, can be added later without breaking existing callers.

If you send the same `deduplication_key` twice, the second one is ignored, so retrying is safe. `source` records which policy and event led to the suggestion, so the PM can see why they got it. Each suggestion is accepted or rejected on its own; one bad item doesn't fail the rest.

The `type` must be one of the actions Veda can already run, and the activity's type tells you which one:

Activity type	Suggestion type
`email`	`messaging.email.send_email`
`calendar`	`scheduling.calendar.create_event`
`slack_teams`	`messaging.slack.post_update` or `messaging.teams.post_update`, depending on what the client has connected
`pm_tool`	`project.ticketing.create_ticket` or `project.ticketing.update_ticket`
`certinia_stage`	`project.certinia.update_stage`
`certinia_raid_create`	`project.certinia.create_raid_record`
`certinia_raid_update`	`project.certinia.update_raid_record`
`insight_content`, `generate_document`	None yet: Veda has no matching action

Each payload follows the schema Veda already uses for that type; we'll share the exact schemas once the list is agreed.

We need from you: does this shape work for you?

4. Who receives a suggestion?

Activities are written for roles, but people read suggestions. Example: the "Post Slack update" activity is for the PM role. On Coca Cola, Anna is the PM. If Anna leaves next week and Mark takes over, who should see a suggestion that is still waiting?

We haven't decided this yet. Two ways it could work:

- **You send the role**, and Veda finds who holds it on that project, now and after the team changes. In the example, the suggestion moves from Anna to Mark by itself.
- **You send the people**, chosen from the team in the context. You decide exactly who sees it, but you also have to follow team changes yourselves.

We need from you: which one fits the orchestrator better, and whether you ever need to address a specific person rather than a role.

5. Which shared lists do we agree on, and what are the exact words?

A lot of this contract is built from short keys that both sides have to spell the same way. Example: the admin picks "Health scan" in the policy form and we store `project.health.status_changed`. When you detect a health change, you look for policies with that key. If you call it `health.changed` instead, the policy never fires and nobody sees an error. The same goes for activity types, suggestion types, event names and the rest.

Our proposal: we keep one catalog in Veda as the single source of truth, the same one the Veda UI uses, and give you an endpoint to read it. Every key is lowercase and never changes once it's used. Labels are only for display and can change at any time. Adding a value is always safe; renaming or removing one needs both teams to agree first. If you send us a key that isn't in the catalog, we reject that item with a clear error instead of guessing.

The lists we see today:

List	Examples	Where it's used
Trigger types	`callpilot.call.ended`, `project.methodology.stage_changed`, `project.health.status_changed`, `agent.chat.message_sent`	When a policy applies
Trigger reasons	Depend on the trigger, e.g. `low_health_score`, `health_declining`	The policy's "reason"
Activity types	`email`, `calendar`, `slack_teams`, `pm_tool`, `certinia_stage`, `certinia_raid_create`, `certinia_raid_update`, `insight_content`, `generate_document`	Each activity in a policy or stage
Suggestion types	`messaging.email.send_email`, `scheduling.calendar.create_event`, `messaging.slack.post_update`, `messaging.teams.post_update`, `project.ticketing.create_ticket`, `project.ticketing.update_ticket`, `project.certinia.update_stage`, `project.certinia.create_raid_record`, `project.certinia.update_raid_record`, `project.methodology.change_stage`	The `type` of every suggestion you send (topic 3)
Event types	The events in topic 1, e.g. `project.methodology.assigned`, `project.methodology.stage_changed`	What we tell you happened (topic 1)
Target kinds	`project_role`, `user`	Who a suggestion is for (topic 4)

List	Examples	Where it's used
Source kinds	`operating_model.policy`, `operating_model.stage`	Why a suggestion was made, in `source` (topic 3)
Methodology types	`one_time`, `recurring`	How a methodology runs

We'd offer one endpoint for the whole catalog, `GET /internal/orchestration/catalog`. It is the same for every client and only changes when we deploy, so you can cache it. Example response:

```
{
  "trigger_types": [{
    "key": "project.health.status_changed", "label": "Health scan",
    "reasons": [
      {"key": "low_health_score", "label": "Health score is low"},
      {"key": "health_declining", "label": "Health is declining"}
    ]
  }],
  "activity_types": [
    {"key": "slack_teams", "label": "Slack/Teams", "suggestion_types":
["messaging.slack.post_update", "messaging.teams.post_update"]}
  ],
  "suggestion_types": [
    {"key": "messaging.slack.post_update", "label": "Post Slack update",
"payload_schema": {"…": "…"}}
  ],
  "event_types": [{"key": "project.methodology.stage_changed", "label": "Stage
changed"}],
  "target_kinds": ["project_role", "user"],
  "source_kinds": ["operating_model.policy", "operating_model.stage"],
  "methodology_types": ["one_time", "recurring"]
}
```

The values above are shortened.

Trigger types and reasons should only list what you can really detect, otherwise admins set up policies that never fire. A policy can also have no trigger and only a written description, such as "Before any client demo, check the latest RAID log"; then it's up to you to work out when it applies.

We need from you: confirm these lists and names, or tell us what is missing or named differently on your side, so we can document them as final.

6. How do stage changes work?

The current stage decides which activities apply. Example: Coca Cola is in "Discovery and Scope". On today's call the client signed off the scope, so the project should move to "Delivery", and the next stage's activities should start.

Our proposal: a project admin moves the stage in Veda. The orchestrator can suggest the move with the type `project.methodology.change_stage`, citing the evidence, such as "Scope signed off on the call of

Nov 18". Only project admins can apply it, so the suggestion should go to team members with `access_level` 400 or more, or to the project owner.

We need from you: does this work for you? Will you notice stage changes yourselves, for example from calls, or only react when a person moves the stage?

Background

Who does what

	Veda (our team)	AI team
Roles, policies, methodologies	Store them; admin UI	Read them
Noticing that something happened	Publishes the events it owns	Detects and interprets
Deciding what to suggest, and writing it	Checks that the suggestion is valid	Owns
Who receives a suggestion	Shows it to the right people; turns roles into people if we target roles (topic 4)	Picks the roles or people
Showing and running suggestions (email, Slack, Teams, calendar, Jira, Certinia)	Owns, with the executors that exist today	—
Moving a project to the next stage	Stores it; a project admin applies the move	Suggests it

Words we use

Term	Meaning
Project Team Role	The part someone plays on a project: PM, BA, Designer. Not an access level.
Policy	A reusable rule: a description of what to do, an optional trigger (when it applies), and a list of activities.
Trigger	The kind of event a policy reacts to, such as a health change, plus an optional reason, such as "health score is low".
Methodology	The client's way of running a project, as ordered stages. Publishing freezes a version, and each project uses one version.
Stage	One step of a methodology, such as "Discovery and Scope". It holds activities and can include policies.

Term	Meaning
Activity	One piece of expected work: a type (email, Slack, Jira…), a description, the roles it is for, and optionally an agent and a prompt.
Suggestion	What you send us: a proposed action on a project, which a person applies or dismisses.

Technical details to settle

- **Access:** your own revocable key per environment, sent as `X-Service-Key`, limited to reading data and writing suggestions, and to the clients you may see.
- **Tracing:** send `X-Request-Id` and we log it, so one request can be followed across both systems.

Open questions

Tracker

#	Question	Why it matters	Recommendation	Owner	Status
29	Which runtime parts must be ready for the December demo?	Sets the scope of the first version. From the Uinno call: it is unknown whether stage changes and the stage display (#19, #20) are in this version.	Stage card and suggestions. Deviations and adherence are a future phase.	Saar, Nadia	Open
14	What should uploading a methodology document do beyond storing it? Will the AI team extract stages and activities from it? Do we need document upload in the first version at all?	MOON-2392 only stores the file "for later processing" and puts extraction out of scope; meetings 1 and 2 only say the AI team will use it. Nothing says what the user gets after uploading.	Until this is answered, upload only stores the file. Turning extracted content into methodology changes is a future phase (see the Future phase tab).	AI team, Saar	Open
15	What does the "Download template" file contain, and in what format? Do we need it in the first version at all?	Yulia: the user downloads the template, fills it in and uploads it, so the document is structured and easy to process. Its format and content are	We serve it as a static file once it is written; no endpoint generates it. Until then, hide the button.	Saar	Open

#	Question	Why it matters	Recommendation	Owner	Status
		not defined yet.			
16	Is "Engage with Agent" in the first version?	Figma shows an "Engage with Agent" chat next to the methodology editor, but no story describes it. Yulia expects it to be wanted in v1 and to work like the Workflow editor: the agent changes the editor's local state, and the user saves separately.	If it is in v1, it is frontend and agent work: the frontend sends the current structure to the agent, and the backend needs nothing beyond the normal save. Saar decides whether it is in v1.	Saar	Open
9	Should a change to a methodology reach the projects that already use it, and when? Example: "Acme ERP Rollout" is in stage 2, "Discovery and Scope". An admin deletes that stage from the methodology and saves. What happens to the project?	MOON-2391 and MOON-2393 ask for Draft and Active status, versions and a version pinned per project, but Figma has none of it. Saar asked whether methodology changes can flow into attached projects, which conflicts with pinning.	Ask Saar to pick one: • **A. Live.** Save applies to every project at once. No draft, versions or Publish, as in Figma today. Deleting a stage that projects are in is blocked until they are moved. Cheapest, but every edit changes running projects immediately. • **B. Draft, then Publish to all.** Edits stay in a draft. Publish applies them to every project at once and first asks where to move projects whose stage was removed. UI: a Publish	Uinno, Saar	Open

#	Question	Why it matters	Recommendation	Owner	Status
		Keeping versions only in metadata while changes apply automatically is not versioning: it costs the same and protects nothing. Design waits for this answer.	button and a status badge (canvas screens 1–3). • **C. Draft, Publish, projects pinned.** Like B, but each project stays on its version until a project admin upgrades it (screens 4–6). Most control, most UI. We recommend B or C. With B, changes still reach projects, as Saar asked, but only when an admin publishes, and removed stages are handled in one place; versions stay internal (audit and orchestrator), with no version picker in Project Settings. C adds per-project control at the cost of more UI (see #18).		
18	How does a project move to a newer methodology version? Depends on #9.	Only option C of #9 pins a project to a version, so only C needs an upgrade path. With A (live edits) there are no versions, and with B every Publish already moves all projects.	If #9 is A or B: close this row. If #9 is C: a project admin re-selects the methodology manually, with a "v4 available" notice in Project Settings.	Saar	Open
8	Do policies and methodologies need archive and restore, or is a normal delete enough? Example: an admin no longer wants the	MOON-2391 and MOON-2394 ask for archive, restore, an Active / Archived status filter and who	Delete only, as in Figma. A policy used by a methodology, or a methodology used by a project, can't be deleted: the confirmation lists where it is used. Add archive later if users ask for it.	Saar	Open

#	Question	Why it matters	Recommendation	Owner	Status
	"Low health alert" policy. Do they archive it, so it is hidden but can be restored later, or delete it?	archived it and when. The Policies list and Methodologies list show only edit and delete, and neither Uinno nor Design remembers agreeing on archive. Archive adds a status, a filter, a badge, Restore and new screens.			
30	What happens when a policy is edited from inside the methodology editor? Example: the "New Project Kickoff" policy is used by two methodologies, and an admin changes its activities while editing one of them.	Policies are shared: the editor picks them from the policy library and never creates them. Editing a copy per methodology would confuse users ("copy 1", "copy 2"); editing the shared policy changes every methodology that uses it.	Edit the shared policy, not a copy. The editor warns that the change applies wherever the policy is used ("Used in 2 methodologies"). Agreed on the Uinno call; Saar to confirm.	Saar	Open
7	How should the Create Policy form let users choose between a trigger and written instructions?	Meeting 2 agreed a policy needs a trigger, a description, or both, but the	Split the two jobs. Description stays a short summary of the policy. A radio button picks how the policy works: **Trigger** (pick a trigger and reason) or **Instructions** (a free-text field,	Saar, Design	Open

#	Question	Why it matters	Recommendation	Owner	Status
	Example: "Low health alert" can be set up with a trigger (project health, reason `low_health_score`), or written as "When the client sounds unhappy on a call, suggest a check-in with the PM". Today the form makes Description required and the trigger optional, so it doesn't show that these are two ways to define a policy.	Figma form requires Description. That causes two problems. First, users think a trigger is always needed and won't see they can just write what they want. Second, "Description" does two jobs: a short summary of the policy, and the instructions the orchestrator follows when there is no trigger. The AI team needs to know which text is instructions.	called "Prompt" in the previous design). The chosen one is required. The backend adds an `instructions` column. Yulia reworks the form once Saar agrees.		
19	On Overview, show the methodology stage, the Certinia stage, or both? Do we need it in the first version at all?	Both are labelled "Stage", so users can't tell them apart.	Option 1: the methodology card when one is assigned; the Certinia card otherwise. Option 2 (Yulia): two widgets, one for the Certinia (Salesforce) stage and one for the methodology stage. How the two relate is still unclear.	Saar	Open
20	Who moves a project to its next methodology stage? Example:	The methodology stage is new and lives in	People and AI suggestions, no Certinia sync. Project admins move the stage from the Overview stage card, to any stage.	Saar	Open

#	Question	Why it matters	Recommendation	Owner	Status
	the project follows "Enterprise Delivery Framework" (5 stages) and is in stage 2, "Discovery and Scope". On today's call the client signed off the scope. Who moves it to stage 3, "Build": the PM by hand, the AI by suggesting the move for a person to accept, or a sync from the Certinia stage in Salesforce?	Veda. It is not the Certinia stage (in Salesforce, e.g. "7/8 Negotiation/Review"): the two lists are defined separately and nothing maps one to the other. The answer decides what we build: a stage picker for people, a `change_stage` suggestion type for the AI, and, for a sync, a per-methodology mapping from Certinia picklist values to stages. It also decides when the orchestrator suggests the next stage's activities. From the Uinno call: no story covers changing or showing the stage. The older design let a user	The AI never moves it directly: it sends a `change_stage` suggestion with evidence ("scope signed off on the call of Nov 18"), and a project admin applies it. A sync is left out because the two stage lists don't line up one to one and each methodology would need its own mapping.		

#	Question	Why it matters	Recommendation	Owner	Status
		change it, a suggestion could propose it, and skipping a stage needed a reason. Activities in a stage run in order: the next one waits for the previous one. Whether v1 shows stages at all is unknown.			
12	An activity can use any tenant agent. What happens to the activity when its agent is deleted?	It sets the foreign key rule (clear the agent, or block the delete) and what the editor and the orchestrator see afterwards.	Clear it and flag the activity in the editor as "Agent removed". If #9 keeps published versions (option B or C), each version keeps the agent's name in its snapshot; with option A (live edits) there is no snapshot and only the flag remains.	Saar	Open
13	An activity can be assigned to several project roles. What happens to the activity when one of its roles is deleted?	It sets the foreign key rule (remove the role from the activity, or block the delete) and what the editor and the orchestrator see afterwards. An activity can end up with no role, and then the	Remove the link; the delete confirmation already says how many activities use the role. Flag an activity left with no role in the editor as "No role". If #9 keeps published versions (option B or C), each version keeps the role's name in its snapshot; with option A (live edits) there is no snapshot and only the flag remains.	Saar	Open

#	Question	Why it matters	Recommendation	Owner	Status
		orchestrator has nobody to assign it to.			
23	When a suggested activity targets a role that several people hold, do we create one suggestion shared by all of them, or one copy per person? Example: "Send kickoff email" is assigned to the PM role, and the project has two PMs, Anna and Bob.	With one copy each, Anna and Bob can both apply, and the client gets two kickoff emails (or two Jira tickets). Preventing that means grouping the copies and cancelling the rest when one is applied. It also decides what Bob sees after Anna acts.	One suggestion with both as assignees, as today for the whole team: it shows in "For you" for Anna and Bob, the first to apply sends the email, and it then shows as applied for both. Agreed on the Uinno call; Saar to confirm.	Saar	Open
24	Can the person a suggestion is assigned to apply it whatever their project access level (Viewer, Contributor, Editor)? Example: Kate holds the BA role but is only a Viewer (100) on the project. The orchestrator assigns her "Create Jira ticket for the gap analysis" through the BA	Today apply needs Editor (300), so Kate gets an error and the suggestion sits until it expires. Roles are assigned regardless of access level, so this will happen for every Viewer or Contributor who holds a role.	—	Saar	Open

#	Question	Why it matters	Recommendation	Owner	Status
	role. Should her Apply work?				
5 (incl. 6)	Which values does each shared list have, and what exactly are they called? The lists: trigger types, trigger reasons, activity types (including what "Certinia suggestion A / B / C" are), suggestion types, event types, target kinds, source kinds and methodology types.	ISS, the frontend and the orchestrator must use the same keys, and renaming them later needs a data migration. Today none of the lists is final: Figma's four trigger labels came from Saar's comments, Figma has no reasons at all, the activity-type keys (`email`, `slack_teams`, …) use an old naming style, and "PM Tool / Jira" could mean creating or updating a ticket.	Agree all the lists with the AI team in topic 5 of AI team: topics to discuss, which has our proposal for each list and the catalog endpoint that serves them. Name triggers and events `<domain>.<entity>.<event>` (as `callpilot.call.ended` in ISS) and reasons in snake_case (e.g. `low_health_score`).	AI team, Saar	Open

Decided

Questions we have answered. Each keeps its original number, so references to it still work.

#	Question	Decision	Decided by
1	Do Project Team Roles need a manual order (drag to reorder), or is	No manual reorder. New roles go last in the list. MOON-2390 asked for reorder, but the	Uinno, Nadia

#	Question	Decision	Decided by
	creation or alphabetical order enough?	Figma design never had it, and Nadia confirmed it is not needed.	
2	No story (MOON-2389 to MOON-2394) covers a user's default project role, but meeting 1 discussed it and Figma has a field for it. Do we build it, and does it work as we assume?	Build it as assumed. Each user has one optional default role (`users.default_project_team_role_id`), set in Personal Information. When the user is added to a project team, it prefills the role select, and whoever adds them can change it. It never changes existing project roles or access. Deleting the role clears it. Saved with a direct Supabase write.	Uinno
3	How does an admin set or change another user's default project role?	Design (Yulia) is adding it to the users screens; the frontend builds it once the screen is ready. The self-service field in Personal Information ships first. No backend work: users RLS already lets tenant admins update the column.	Uinno, Design
4	What does "Project roles" in Project Settings → Methodology mean?	Remove it. It is left over from an older design and no story uses it. Design (Yulia) removes it from Figma; we build nothing for it.	Uinno, Design
10	What does a Recurring methodology mean at runtime?	A One-Time methodology stops after its last stage. A Recurring one starts again at stage 1 after its last stage, like sprints.	Uinno
11	Does the Activity form need a "Guidance / expected outcome" field?	No. Drop the field: it comes from an older design, and the Prompt already covers the expected outcome.	Uinno
17	Who can select a project's methodology: access level 400, or owner and company admin only?	Access level 400, matching who can open Project Settings.	Uinno

Future phase

Everything here is **out of the first version**. The stories exclude it, or no story asks for it. The designs below were written for this doc earlier and are kept as they were, so the work is not lost; revisit them when the phase is planned. The first version may keep the current stage and the stage card in Project IQ (open questions #19, #20, #29), and keeps the internal API for reading context and writing suggestions.

Feature	Why it waits
Deviations and deviation reasons	MOON-2391 and MOON-2393 exclude "deviation detection" (`base.md:386`, `:621`).
Stage-advance check (409 on incomplete required activities)	Only makes sense with deviations. In the first version a stage moves freely.
Adherence score, level and snapshots	Excluded with "adherence tracking" (`:386`, `:621`); the epic starts with data capture only (`:186`).
Activity tracking (`project_methodology_activity_states`)	No story asks for it, and the stories exclude rendering activity progress (`base.md:387`). Its only first-version reader would have been the orchestrator; whether it needs it is a question for the AI team.
Manual activity marking (activities drawer, `complete_activity` suggestion)	No story and no Figma. Builds on activity tracking.
Orchestrator autonomy (direct stage and activity-state writes)	The initial release is human-in-the-loop (`:141`).
AI-proposed changes to methodologies and policies	The epic lists the approval workflow as a future phase (`:165`).
Methodology extraction from documents	Out of scope for MOON-2392 (`:485`); open question #14.
Engage with Agent	No story; open question #16.
Suggestion Sets	Excluded by every story (`:310`, `:382–384`, `:489`, `:622`, `:792`).
History view (project, methodology, policy)	No story or Figma frame shows history; the stories only require recording it. A cursor-paginated `GET /projects/{project_id}/methodology/history` and History panels for methodologies and policies read `operating_model_audit_trail` filtered by project or entity.
Agent prompt suggestion (`agent.run_prompt`)	No story asks for it.

Feature	Why it waits
Visibility standard, later steps (scope tables, audience reuse, events)	No MOON-2389 story needs them. They touch shared ticketing and callpilot code, so each is its own ticket.

Deviations

A deviation records why a required activity was skipped. Fields come from the Method Deviation modal: reason (required), notes (optional), document (required).

Tables

Table	Columns	Constraints and indexes
`project_methodology_deviations`	`project`, `version_id`, `stage_id`, `activity_id`, `reason_id` → client_deviation_reasons, `notes text`, `document_path`, `document_filename`, `document_size`, `created_at`, `created_by`	Index (project, created_at desc). Append-only; a correction is a new deviation.
`client_deviation_reasons`	`name`, `name_key`, `position`, `is_active`	UNIQUE (tenant, name_key). Org-managed, seeded with defaults such as "Client refused scope".

API

Method and path	Guard	Behavior
`POST /projects/{project_id}/methodology/deviations`	EditorProjectSummary	Multipart `activity_id`, `reason_id`, `notes`, `file` (required). Marks the activity skipped.
`GET /projects/{project_id}/methodology/deviations`	ViewerProjectSummary	Deviations with signed document links.
`GET, POST, PATCH, DELETE /deviation-reasons`, `PUT …/order`	Read: AuthenticatedUser. Write: RequireTenantAdmin	Org-managed reason list.

- The document is uploaded first, to `{tenant}/projects/{project}/deviations/{upload_uuid}/{filename}` in the `operating-model` bucket, with a 25 MB limit and the same content checks as methodology documents. Then the RPC inserts the deviation; if it fails, the file is deleted.

- Every deviation writes `project.deviation.recorded` (`{activity_id, reason_id}`), which the orchestrator can use as a risk signal.
- Permissions: Editor (300+) records deviations; tenant admins manage reasons.
- Frontend: the Method Deviation modal (Reason*, Notes, Document* with `file-picker`) and a deviation-reasons admin section on the Policies page.

Stage-advance check

1. Moving backward, or staying on the same stage, is always allowed.
2. Moving forward: the RPC collects required activities in every stage the move passes (from the current stage up to, not including, the target) whose state is not completed, skipped or not applicable.
3. If that list is not empty, `PUT …/methodology/stage` returns 409 `operating_model.project_methodology.required_activities_incomplete` with `meta.activities[]` (stage name, activity name, activity id).
4. The frontend opens the Method Deviation modal once per listed activity. Each Confirm sends `POST …/methodology/deviations`, which also marks the activity skipped.
5. The frontend retries the stage change, which now succeeds.

When autonomy is on and the orchestrator moves the stage directly, the move is allowed and the missing required activities count as overdue; only a person can write a deviation reason.

Adherence

The Project IQ frame in Figma shows "Medium adherence" next to the stage. In the first version that label stays hidden.

- **Due** = required activities in stages before the current stage, counted per (stage, activity).
- **Done** = due activities that are completed. **Justified** = due activities skipped with a deviation.
- **Score** = (done + justified) / due × 100. With no due activities there is no score.
- **Level**: High at 85 or above, Medium from 60 to 84, Low below 60 (placeholder thresholds for Product to confirm).
- Deviations count as justified but are reported separately, so a project that skips everything with reasons is still visible.
- `get_project_methodology_adherence(p_project)` computes it from the pinned snapshot and the state rows. Every RPC that changes the stage, an activity state or a deviation recomputes it and stores `adherence_score numeric(5,2)` and `adherence_level` on `project_methodology`. A level change writes `project.methodology.adherence_level_changed` (`{from_level, to_level, score}`).
- A daily workflow-service beat job calls `snapshot_project_methodology_adherence()` to fill `project_methodology_adherence_snapshots` (`project`, `snapshot_date`, `version_id`, `stage_id`, `required_due`, `required_done`, `required_justified`, `score`, `level`; PK (project, snapshot_date)).
- API: `GET /projects/{project_id}/methodology/adherence` (ViewerProjectSummary) returns the current score and the daily trend; the methodology summary gains `adherence {score, level}`. The stage card shows a dot with the level and the score in a tooltip.

Activity tracking

Records, per project, which methodology activities are done. Deviations, adherence, manual marking, orchestrator autonomy and the "next activity" text on the stage card ("Stage 2/5 · Discovery and Scope: Gap analysis") all build on it.

Why a table and not applied suggestions. "Completed" could be read as "an applied suggestion exists with that activity's scope", but applied suggestions don't last: `cleanup_suggestions` in workflow-service deletes every suggestion past `expires_at`, applied ones included, and deleting a project or regenerating a call removes them too. A durable record needs its own table.

Table `project_methodology_activity_states`

Column	Notes
`project`	→ project CASCADE
`stage_id`, `activity_id`	Ids from the pinned snapshot; stable across versions, so states carry over on an upgrade for activities present in both
`status`	pending / completed at first; in_progress, skipped and not_applicable come with manual marking and deviations
`changed_at`, `changed_by`	Who and when
`suggestion_id`	→ suggestions SET NULL; the suggestion whose apply completed it

PK (project, stage_id, activity_id). `actor_type` and `evidence jsonb` are added with the writers that need them: orchestrator autonomy (`ai`, `{source_type, source_id, summary}`), manual marking (a note) and deviations (the deviation id).

How it fills

- A `SECURITY DEFINER` trigger, `AFTER UPDATE OF status ON suggestions WHEN (NEW.status = 'applied')`, reads the suggestion's `operating_model.stage` and `operating_model.activity` scopes and upserts `completed` in the same transaction as the apply.
- Every change writes `project.activity_state.changed` to `operating_model_audit_trail`, so the orchestrator can stop suggesting completed activities.
- The context API returns `activity_states: [{stage_id, activity_id, status, changed_at}]`.
- The stage card shows the first activity of the current stage, by position, that is not completed.
- Read access: Viewer (100), with the project methodology.

Manual activity marking

From	To	Who	Evidence
pending	in_progress / completed / not_applicable	Editor (300+), or the AI through a `project.methodology.complete_activity` suggestion (`{stage_id, activity_id, evidence}`)	Optional note; for AI the source call, ticket or suggestion id
pending / in_progress	skipped	Only by recording a deviation	The deviation id
any	pending	Editor (300+)	Optional note

- API: `GET /projects/{project_id}/methodology/activities` (stages and activities from the pinned snapshot, each with its state) and `PUT /projects/{project_id}/methodology/stages/{stage_id}/activities/{activity_id}/state` (`{status, evidence?}`, EditorProjectSummary).
- Frontend: a "Methodology activities" drawer opened from the stage card, listing stages and activities with their status (no Figma yet).

Orchestrator autonomy

A tenant setting, `orchestrator_autonomy` in `subscriber_org_defaults`, lets the orchestrator write project state directly instead of through suggestions. Off by default.

Internal endpoint	Scope	Behavior
`PUT /internal/orchestration/projects/{project_id}/stage`	`orchestration:write`	`{stage_id, evidence: {source_type, source_id, summary}, confidence}`. Recorded with `actor_type = ai`. Adds a `current_stage_source` column (user / ai, default user) to `project_methodology`, because an AI move has no person in `current_stage_set_by`.
`PUT /internal/orchestration/projects/{project_id}/stages/{stage_id}/activities/{activity_id}/state`	`orchestration:write`	`{status: in_progress | completed | not_applicable, evidence}`. `skipped` is reserved for human deviations.

- Errors: 403 `operating_model.orchestration.autonomy_disabled` while the setting is off; 409 `operating_model.project_methodology.not_assigned`; 422 `operating_model.project_methodology.stage_not_in_version`.
- Idempotency: setting the same target state again is a no-op; an optional `Idempotency-Key` header is remembered for 24 hours.

AI-proposed changes to methodologies and policies

The epic requires that "AI-derived methodology or policy refinements must be proposed for client approval before becoming active" (`base.md:136`, `:178`).

- **Producers**: document extraction (after the `uploaded` event), the "Engage with Agent" chat in the editor, and later refinements learned from transcripts.
- **Table** `operating_model_proposals`: `entity_type` (methodology / policy), `entity_id` nullable (null when a new policy is proposed), `base_revision int`, `proposed jsonb`, `summary text`, `source` (operating_model.document / agent.chat / callpilot.call), `source_ref jsonb`, `status` (pending / accepted / rejected / superseded), `created_by_service`, `decided_at`, `decided_by`, `decision_note`.
- **Shape**: `proposed` uses the same body as `PUT …/structure` for a methodology, or `PUT /client-policies/{id}` for a policy. `base_revision` records the revision the AI read.
- **API**: internal `POST /internal/orchestration/proposals` (`proposals:write`, `{entity_type, entity_id?, base_revision, proposed, summary, source, source_ref}`); public `GET /client-methodologies/{id}/proposals` and `POST /operating-model-proposals/{proposal_id}/accept` and `/reject` (RequireTenantAdmin).
- **Review**: the editor shows a banner "AI proposed changes" and draws the diff on the canvas, with Accept and Reject.
- **Accept**: RPC `accept_operating_model_proposal` applies the proposal through the structure RPC. If `base_revision` is older than the methodology, it returns 409 `operating_model.proposal.stale`; the user asks the agent again or chooses "Replace draft". Document extraction is slow, so its proposals are often stale.
- **Activation**: accepting a methodology proposal changes only the draft, and Publish stays a separate human step. Policies have no draft, so accepting a policy proposal writes a new revision; the acceptance is the approval.
- **Superseding**: a new pending proposal for the same entity supersedes older pending ones.
- **Events**: `operating_model.proposal` created, accepted, rejected, superseded.
- **AI triggers**: once proposals exist, the orchestrator can suggest a trigger for a policy that has none by posting a policy proposal.

Methodology extraction from documents

In the first version an uploaded document is only stored and announced with `operating_model.methodology_document.uploaded`. Extraction would read stages, activities, roles and policies out of the file and return a methodology proposal (above). A `client_methodology_documents.extraction_status` column (not_requested / pending / completed / failed), updated by the AI team through an internal endpoint, would track it. Whether and when it happens is open question #14.

Engage with Agent

Figma shows an "Engage with Agent" chat next to the methodology editor. It would open `assistant-panel-layout` with a methodology agent; `chat-assistant.directive.ts` assumes a

project context today and needs an org-level mode. If the agent changes the methodology, the changes go through proposals. Whether it is in the first version is open question #16.

Suggestion Sets

Only the epic describes them: guidance and actions linked to a methodology's activities ("Linked suggestion set or flow", `base.md:48`, also `:145`, `:160`), applied when a project selects that methodology while PMs keep permitted adjustments (`:124`). Figma has a "Suggestions Sets" tile and a "Suggestions set" select in Project Settings, with no editor screens. Both stay hidden; projects keep today's per-project `suggestion_settings`.

An earlier draft of this doc modelled a set as a named copy of today's suggestion settings. It does not match the epic's activity-level idea and needs product input, but is kept as a starting point:

- `suggestion_sets`: `name`, `name_key`, `description`, `volume` (same enum as `suggestion_settings.volume`), `categories jsonb` (same shape as `suggestion_settings.categories`), `enabled_activity_types text[]` (empty means all), `relay_agent_overrides jsonb`, `is_default bool`, `status` (active / archived).
- `project_suggestion_sets`: `project` PK → project CASCADE, `suggestion_set_id` → suggestion_sets RESTRICT, `selected_at`, `selected_by`.
- Resolution: the selected set, otherwise the project's own override row in `suggestion_settings`, otherwise the org row. `get_suggestion_settings_for_project` and `apply_suggestion_settings_trigger` follow that order.
- API: `GET`, `POST`, `PUT`, archive and `DELETE` on `/suggestion-sets`; `PUT /projects/{project_id}/suggestion-set` with `{suggestion_set_id | null}`.
- Optional: a methodology could carry `default_suggestion_set_id`, preselected in Project Settings.

Visibility standard: later steps

The first version adds the rules, the `audience` target, the RLS policy and the daily cleanup (Architecture → Visibility standard). These steps finish the job and can be done one at a time.

Step	What changes	Why it waits
One scope shape	`suggestion_scopes` and `subscriber_event_scopes` get the same PK, `(owner, entity_type, entity_id)`, and the NULL wildcard goes. "Shows in every collection" becomes an explicit flag. `_apply_scope_filter` and `count_unseen_suggestions` change with it.	Touches ticketing and callpilot suggestions; no gain for this feature.
Reuse audiences	An audience is keyed by a hash of its sorted members, unique per project, and rows targeting the same people share it. Cleanup then counts every row that points to it.	Only matters if one audience per row grows the tables. Measure first.

Step	What changes	Why it waits
Incremental cleanup	The daily step checks only audiences whose rows were deleted since the last run, in batches, instead of scanning all of them.	The full scan is cheap at today's volume.
Events use audiences	`subscriber_events` producers state an intent and go through the `AudienceResolver`, like suggestions. Today they set `project` or `user` by hand.	No event needs a group audience yet.
Role-based targets	A target that follows a role ("everyone who is BA on this project", re-resolved when the team changes) instead of a list frozen at creation.	v1 freezes the list at ingestion, which matches what a suggestion needs.